Exploratory Data Analysis

---

1. Import packages
1. Loading data with Pandas
2. Descriptive statistics of data
3. Data visualization

---

1. Import packages

```
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Shows plots in Jupyter notebook
%matplotlib inline

# Set plot style
sns.set(color_codes=True)
```

---

2. Loading data with Pandas

We need to load `client_data.csv` and `price_data.csv` into individual dataframes so that we can work with them in Python

```
client_df = pd.read_csv('./client_data.csv')
price_df = pd.read_csv('./price_data.csv')
```

Let's look at the first 3 rows of both dataframes to see what the data looks like

```
client_df.head(3)
```

```
                                  id                      channel_sales  \
0  24011ae4ebbe3035111d65fa7c15bc57  foosdfpfkusacimwkcsosbicdxkicaua
1  d29c2c54acc38ff3c0614d0a653813dd                           MISSING
2  764c75f661154dac3a6c254cd082ea7d  foosdfpfkusacimwkcsosbicdxkicaua

   cons_12m  cons_gas_12m  cons_last_month  date_activ    date_end  \
0         0         54946                0  2013-06-15  2016-06-15
1      4660             0                0  2009-08-21  2016-08-30
2       544             0                0  2010-04-16  2016-04-16

  date_modif_prod date_renewal  forecast_cons_12m  ...  has_gas  imp_cons  \
0      2015-11-01   2015-06-23               0.00  ...        t       0.0
```

```
1      2009-08-21   2015-08-31             189.95  ...        f       0.0
2      2010-04-16   2015-04-17              47.96  ...        f       0.0

   margin_gross_pow_ele  margin_net_pow_ele  nb_prod_act  net_margin  \
0                 25.44               25.44            2      678.99
1                 16.38               16.38            1       18.89
2                 28.60               28.60            1        6.60

  num_years_antig                        origin_up  pow_max  churn
0               3  lxidpiddsbxsbosboudacockeimpuepw   43.648      1
1               6  kamkkxfxxuwbdslkwifmmcsiusiuosws   13.800      0
2               6  kamkkxfxxuwbdslkwifmmcsiusiuosws   13.856      0

[3 rows x 26 columns]
```

With the client data, we have a mix of numeric and categorical data, which we will need to transform before modelling later

```
price_df.head(3)
```

```
                                id  price_date  price_off_peak_var  \
0  038af19179925da21a25619c5a24b745  2015-01-01            0.151367
1  038af19179925da21a25619c5a24b745  2015-02-01            0.151367
2  038af19179925da21a25619c5a24b745  2015-03-01            0.151367

   price_peak_var  price_mid_peak_var  price_off_peak_fix  price_peak_fix  \
0             0.0                 0.0           44.266931             0.0
1             0.0                 0.0           44.266931             0.0
2             0.0                 0.0           44.266931             0.0

   price_mid_peak_fix
0                 0.0
1                 0.0
2                 0.0
```

With the price data, it is purely numeric data but we can see a lot of zeros

---

3. Descriptive statistics of data

Data types

It is useful to first understand the data that you're dealing with along with the data types of each column. The data types may dictate how you transform and engineer features.

```
client_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14606 entries, 0 to 14605
Data columns (total 26 columns):
```

```
 #   Column                           Non-Null Count  Dtype
---  ------                           --------------  -----
 0   id                               14606 non-null  object
 1   channel_sales                    14606 non-null  object
 2   cons_12m                         14606 non-null  int64
 3   cons_gas_12m                     14606 non-null  int64
 4   cons_last_month                  14606 non-null  int64
 5   date_activ                       14606 non-null  object
 6   date_end                         14606 non-null  object
 7   date_modif_prod                  14606 non-null  object
 8   date_renewal                     14606 non-null  object
 9   forecast_cons_12m                14606 non-null  float64
 10  forecast_cons_year               14606 non-null  int64
 11  forecast_discount_energy         14606 non-null  float64
 12  forecast_meter_rent_12m          14606 non-null  float64
 13  forecast_price_energy_off_peak   14606 non-null  float64
 14  forecast_price_energy_peak       14606 non-null  float64
 15  forecast_price_pow_off_peak      14606 non-null  float64
 16  has_gas                          14606 non-null  object
 17  imp_cons                         14606 non-null  float64
 18  margin_gross_pow_ele             14606 non-null  float64
 19  margin_net_pow_ele               14606 non-null  float64
 20  nb_prod_act                      14606 non-null  int64
 21  net_margin                       14606 non-null  float64
 22  num_years_antig                  14606 non-null  int64
 23  origin_up                        14606 non-null  object
 24  pow_max                          14606 non-null  float64
 25  churn                            14606 non-null  int64
dtypes: float64(11), int64(7), object(8)
memory usage: 2.9+ MB
```

```
price_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 193002 entries, 0 to 193001
Data columns (total 8 columns):
 #   Column              Non-Null Count   Dtype
---  ------              --------------   -----
 0   id                  193002 non-null  object
 1   price_date          193002 non-null  object
 2   price_off_peak_var  193002 non-null  float64
 3   price_peak_var      193002 non-null  float64
 4   price_mid_peak_var  193002 non-null  float64
 5   price_off_peak_fix  193002 non-null  float64
 6   price_peak_fix      193002 non-null  float64
 7   price_mid_peak_fix  193002 non-null  float64
dtypes: float64(6), object(2)
memory usage: 11.8+ MB
```

You can see that all of the `datetime` related columns are not currently in datetime format. We will need to convert these later.

Statistics

Now let's look at some statistics about the datasets

```
client_df.describe()
```

```
           cons_12m  cons_gas_12m  cons_last_month  forecast_cons_12m  \
count  1.460600e+04  1.460600e+04     14606.000000       14606.000000
mean   1.592203e+05  2.809238e+04     16090.269752        1868.614880
std    5.734653e+05  1.629731e+05     64364.196422        2387.571531
min    0.000000e+00  0.000000e+00         0.000000           0.000000
25%    5.674750e+03  0.000000e+00         0.000000         494.995000
50%    1.411550e+04  0.000000e+00       792.500000        1112.875000
75%    4.076375e+04  0.000000e+00      3383.000000        2401.790000
max    6.207104e+06  4.154590e+06    771203.000000       82902.830000

       forecast_cons_year  forecast_discount_energy  forecast_meter_rent_12m
\
count        14606.000000              14606.000000             14606.000000
mean          1399.762906                  0.966726                63.086871
std           3247.786255                  5.108289                66.165783
min              0.000000                  0.000000                 0.000000
25%              0.000000                  0.000000                16.180000
50%            314.000000                  0.000000                18.795000
75%           1745.750000                  0.000000               131.030000
max         175375.000000                 30.000000               599.310000

       forecast_price_energy_off_peak  forecast_price_energy_peak  \
count                    14606.000000                14606.000000
mean                         0.137283                    0.050491
std                          0.024623                    0.049037
min                          0.000000                    0.000000
25%                          0.116340                    0.000000
50%                          0.143166                    0.084138
75%                          0.146348                    0.098837
max                          0.273963                    0.195975

       forecast_price_pow_off_peak      imp_cons  margin_gross_pow_ele  \
count                 14606.000000  14606.000000          14606.000000
mean                     43.130056    152.786896             24.565121
std                       4.485988    341.369366             20.231172
min                       0.000000      0.000000              0.000000
25%                      40.606701      0.000000             14.280000
50%                      44.311378     37.395000             21.640000
75%                      44.311378    193.980000             29.880000
max                      59.266378  15042.790000            374.640000
```

```
       margin_net_pow_ele   nb_prod_act    net_margin  num_years_antig  \
count        14606.000000  14606.000000  14606.000000     14606.000000
mean            24.562517      1.292346    189.264522         4.997809
std             20.230280      0.709774    311.798130         1.611749
min              0.000000      1.000000      0.000000         1.000000
25%             14.280000      1.000000     50.712500         4.000000
50%             21.640000      1.000000    112.530000         5.000000
75%             29.880000      1.000000    243.097500         6.000000
max            374.640000     32.000000  24570.650000        13.000000

            pow_max         churn
count  14606.000000  14606.000000
mean      18.135136      0.097152
std       13.534743      0.296175
min        3.300000      0.000000
25%       12.500000      0.000000
50%       13.856000      0.000000
75%       19.172500      0.000000
max      320.000000      1.000000
```

The describe method gives us a lot of information about the client data. The key point to take away from this is that we have highly skewed data, as exhibited by the percentile values.

```
price_df.describe()
```

```
       price_off_peak_var  price_peak_var  price_mid_peak_var  \
count       193002.000000   193002.000000       193002.000000
mean             0.141027        0.054630            0.030496
std              0.025032        0.049924            0.036298
min              0.000000        0.000000            0.000000
25%              0.125976        0.000000            0.000000
50%              0.146033        0.085483            0.000000
75%              0.151635        0.101673            0.072558
max              0.280700        0.229788            0.114102

       price_off_peak_fix  price_peak_fix  price_mid_peak_fix
count       193002.000000   193002.000000       193002.000000
mean            43.334477       10.622875            6.409984
std              5.410297       12.841895            7.773592
min              0.000000        0.000000            0.000000
25%             40.728885        0.000000            0.000000
50%             44.266930        0.000000            0.000000
75%             44.444710       24.339581           16.226389
max             59.444710       36.490692           17.458221
```

Overall the price data looks good.

3. Data visualization

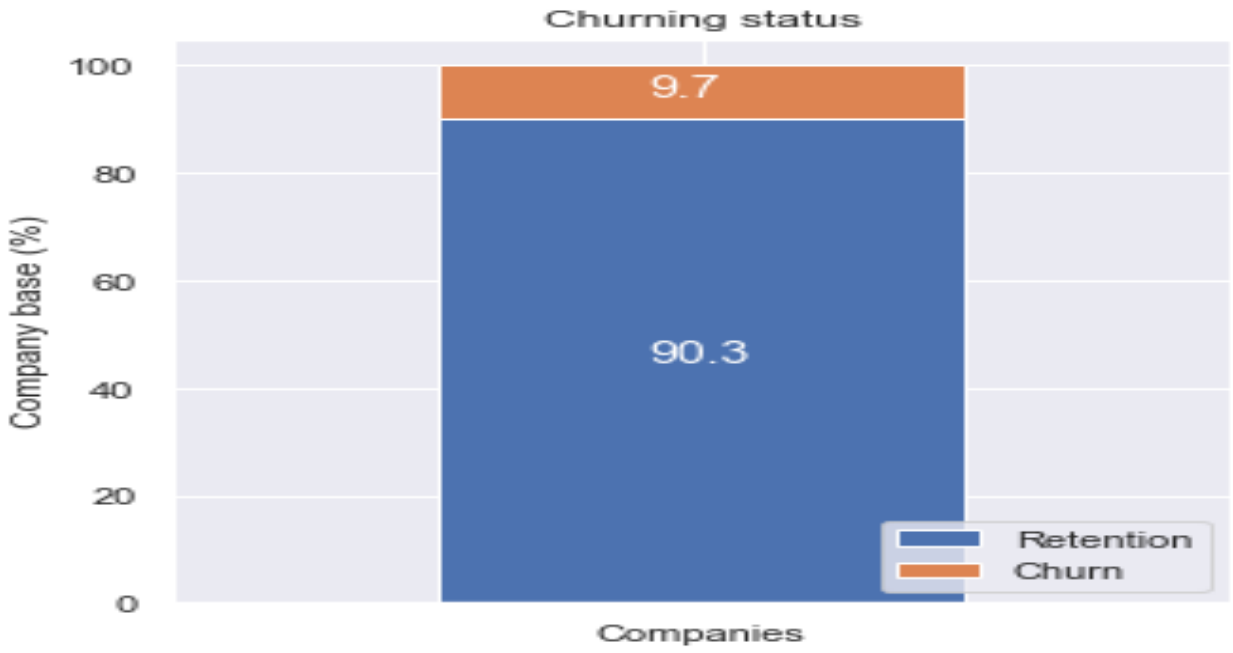
Now let's dive a bit deeper into the dataframes

```
def plot_stacked_bars(dataframe, title_, size_=(18, 10), rot_=0,
legend_="upper right"):
    """
    Plot stacked bars with annotations
    """
    ax = dataframe.plot(
        kind="bar",
        stacked=True,
        figsize=size_,
        rot=rot_,
        title=title_
    )

    # Annotate bars
    annotate_stacked_bars(ax, textsize=14)
    # Rename legend
    plt.legend(["Retention", "Churn"], loc=legend_)
    # Labels
    plt.ylabel("Company base (%)")
    plt.show()

def annotate_stacked_bars(ax, pad=0.99, colour="white", textsize=13):
    """
    Add value annotations to the bars
    """

    # Iterate over the plotted rectanges/bars
    for p in ax.patches:

        # Calculate annotation
        value = str(round(p.get_height(),1))
        # If value is 0 do not annotate
        if value == '0.0':
            continue
        ax.annotate(
            value,
            ((p.get_x()+ p.get_width()/2)*pad-0.05,
(p.get_y()+p.get_height()/2)*pad),
            color=colour,
            size=textsize
        )
```

Churn

```
churn = client_df[['id', 'churn']]
churn.columns = ['Companies', 'churn']
```

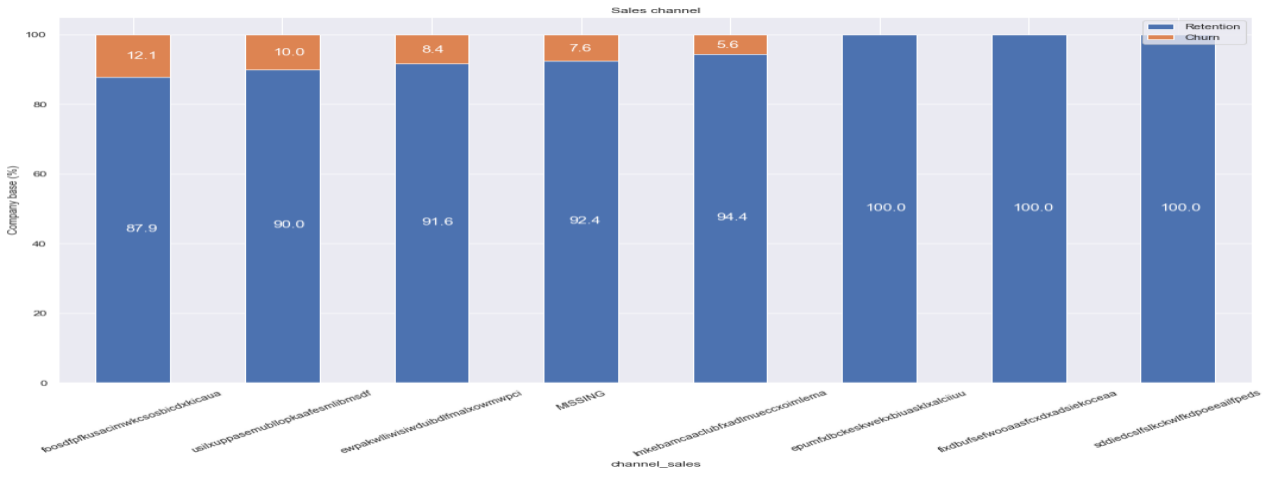
```
churn_total = churn.groupby(churn['churn']).count()
churn_percentage = churn_total / churn_total.sum() * 100

plot_stacked_bars(churn_percentage.transpose(), "Churning status", (5, 5),
legend_="lower right")
```



About 10% of the total customers have churned. (This sounds about right)

Sales channel

```
channel = client_df[['id', 'channel_sales', 'churn']]
channel = channel.groupby([channel['channel_sales'],
channel['churn']])['id'].count().unstack(level=1).fillna(0)
channel_churn = (channel.div(channel.sum(axis=1), axis=0) *
100).sort_values(by=[1], ascending=False)

plot_stacked_bars(channel_churn, 'Sales channel', rot_=30)
```

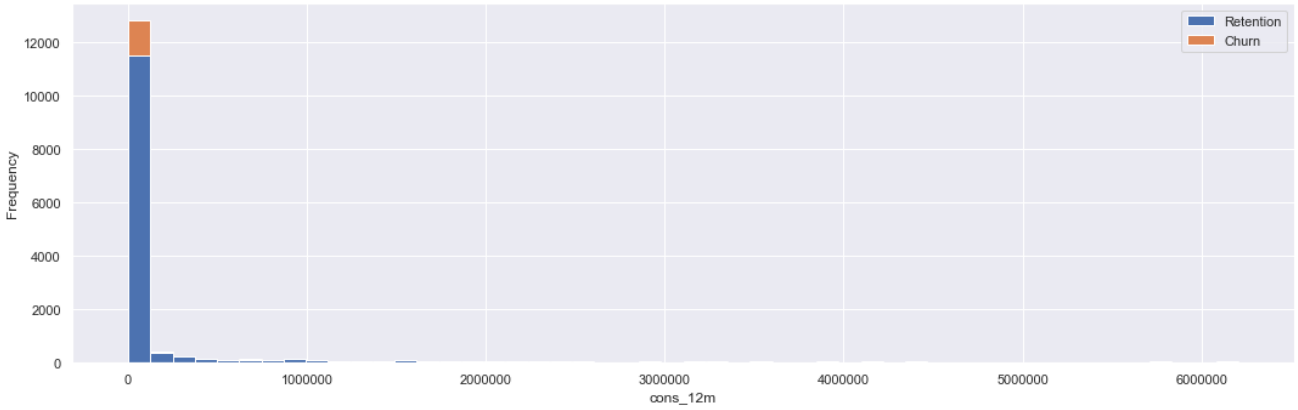


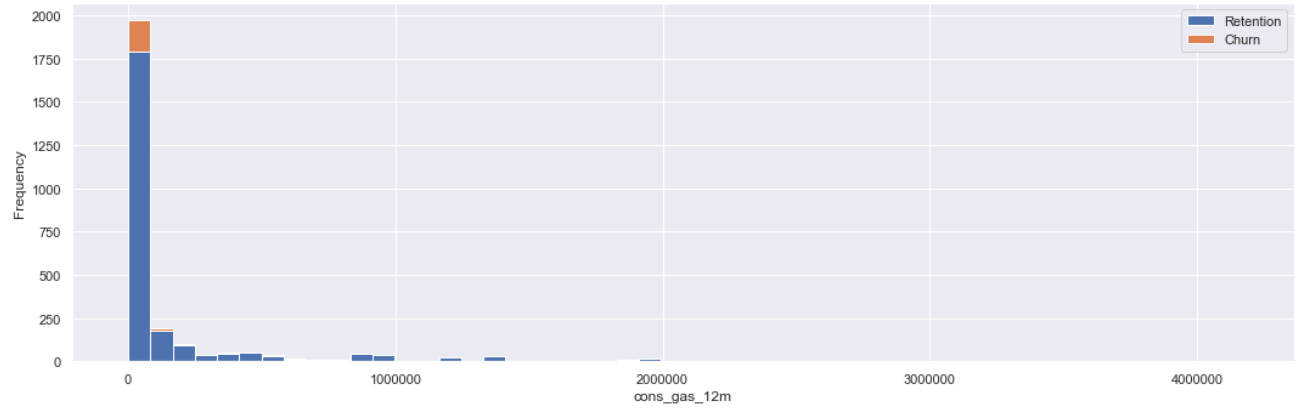
Interestingly, the churning customers are distributed over 5 different values for `channel_sales`. As well as this, the value of `MISSING` has a churn rate of 7.6%. `MISSING` indicates a missing value and was added by the team when they were cleaning the dataset. This feature could be an important feature when it comes to building our model.

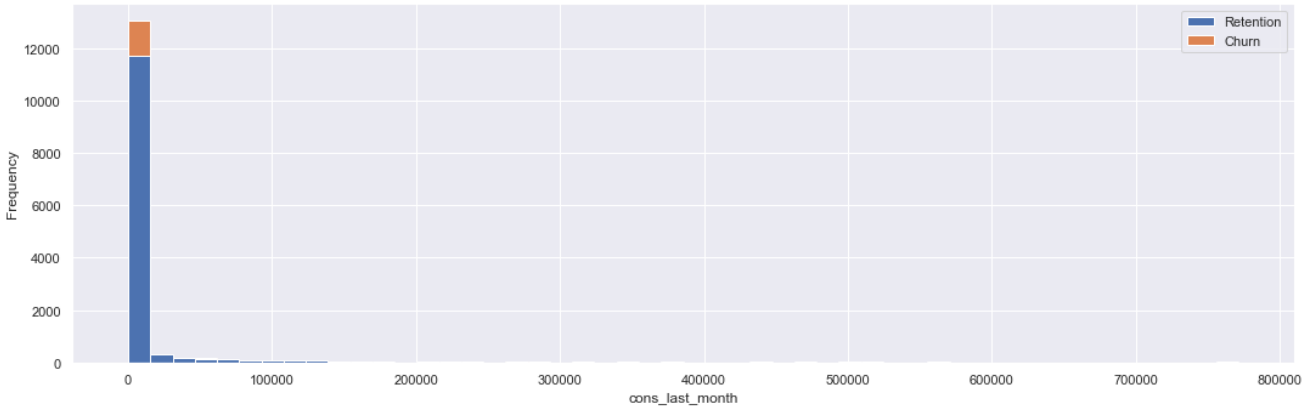
Consumption

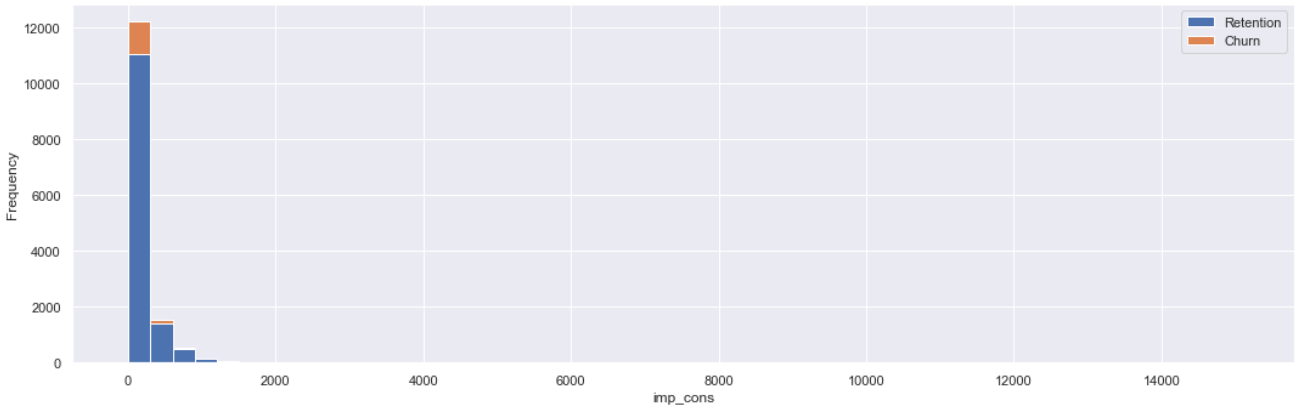
Let's see the distribution of the consumption in the last year and month. Since the consumption data is univariate, let's use histograms to visualize their distribution.

```
consumption = client_df[['id', 'cons_12m', 'cons_gas_12m', 'cons_last_month',
'imp_cons', 'has_gas', 'churn']]

def plot_distribution(dataframe, column, ax, bins_=50):
    """
    Plot variable distribution in a stacked histogram of churned or retained
company
    """
    # Create a temporal dataframe with the data to be plot
    temp = pd.DataFrame({"Retention":
dataframe[dataframe["churn"]==0][column],
    "Churn":dataframe[dataframe["churn"]==1][column]})
    # Plot the histogram
    temp[["Retention","Churn"]].plot(kind='hist', bins=bins_, ax=ax,
stacked=True)
    # X-axis label
    ax.set_xlabel(column)
    # Change the x-axis to plain style
    ax.ticklabel_format(style='plain', axis='x')

fig, axs = plt.subplots(nrows=4, figsize=(18, 25))

plot_distribution(consumption, 'cons_12m', axs[0])
plot_distribution(consumption[consumption['has_gas'] == 't'], 'cons_gas_12m',
axs[1])
plot_distribution(consumption, 'cons_last_month', axs[2])
plot_distribution(consumption, 'imp_cons', axs[3])
```


Retention
Churn
Frequency
0
2000
4000
6000
8000
10000
12000
0
1000000
2000000
3000000
4000000
5000000
6000000
cons_12m


Retention
Churn
Frequency
0
250
500
750
1000
1250
1500
1750
2000
0
1000000
2000000
3000000
4000000
cons_gas_12m


Retention
Churn
Frequency
0
2000
4000
6000
8000
10000
12000
0
100000
200000
300000
400000
500000
600000
700000
800000
cons_last_month

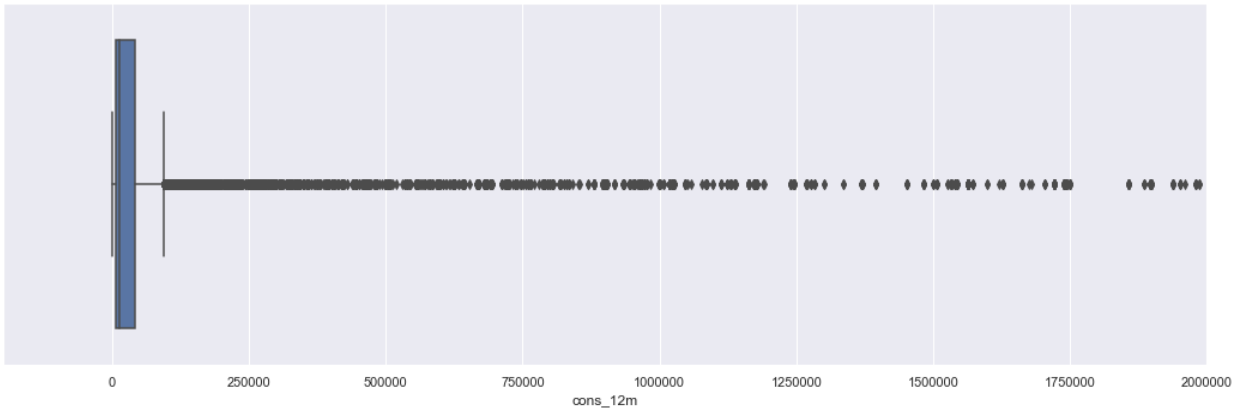

Retention
Churn
Frequency
0
2000
4000
6000
8000
10000
12000
0
2000
4000
6000
8000
10000
12000
14000
imp_cons

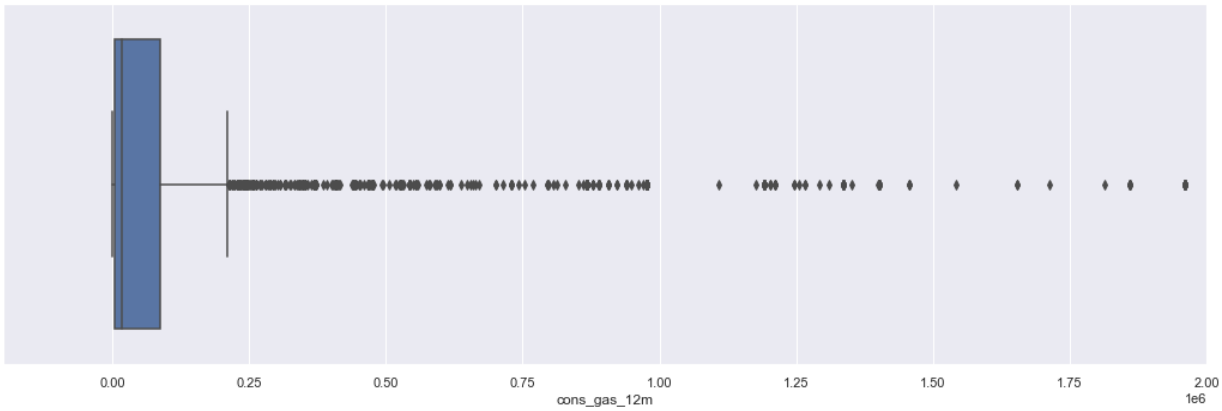
Clearly, the consumption data is highly positively skewed, presenting a very long right-tail towards the higher values of the distribution. The values on the higher and lower ends of the distribution are likely to be outliers. We can use a standard plot to visualise the outliers in more detail. A boxplot is a standardised way of displaying the distribution based on a five-number summary:

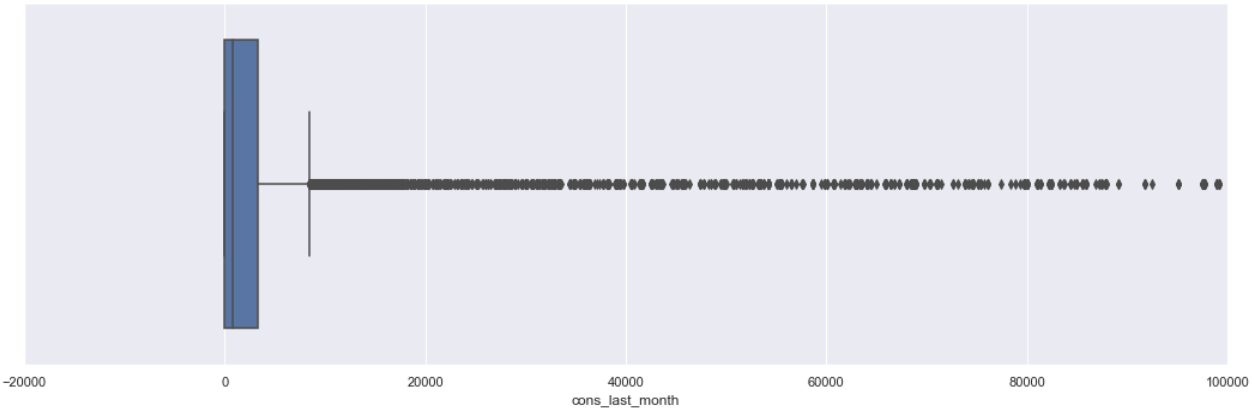
- Minimum
- First quartile (Q1)
- Median
- Third quartile (Q3)
- Maximum

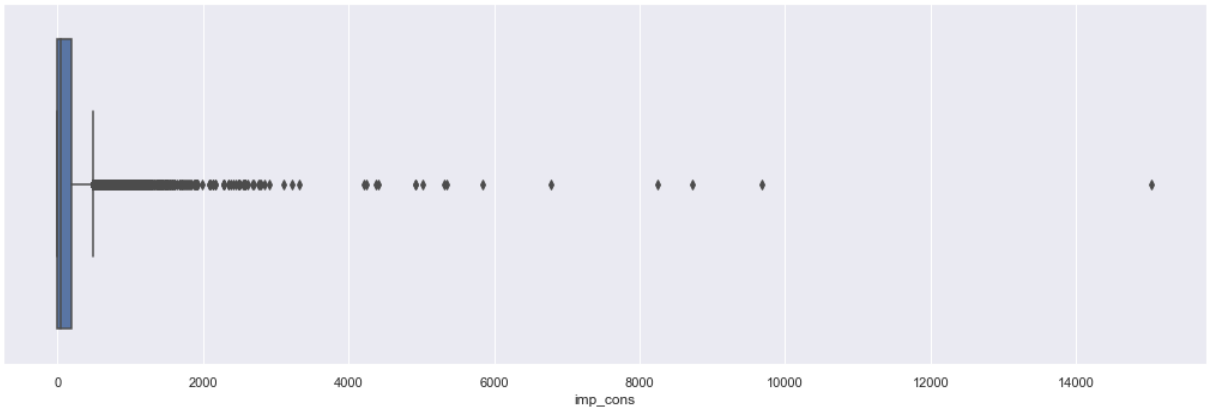
It can reveal outliers and what their values are. It can also tell us if our data is symmetrical, how tightly our data is grouped and if/how our data is skewed.

```
fig, axs = plt.subplots(nrows=4, figsize=(18,25))

# Plot histogram
sns.boxplot(consumption["cons_12m"], ax=axs[0])
sns.boxplot(consumption[consumption["has_gas"] == "t"]["cons_gas_12m"],
ax=axs[1])
sns.boxplot(consumption["cons_last_month"], ax=axs[2])
sns.boxplot(consumption["imp_cons"], ax=axs[3])

# Remove scientific notation
for ax in axs:
    ax.ticklabel_format(style='plain', axis='x')
    # Set x-axis limit
    axs[0].set_xlim(-200000, 2000000)
    axs[1].set_xlim(-200000, 2000000)
    axs[2].set_xlim(-20000, 100000)
    plt.show()
```


0
250000
500000
750000
1000000
1250000
1500000
1750000
2000000
cons_12m


0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
1e6
cons_gas_12m


−20000
0
20000
40000
60000
80000
100000
cons_last_month

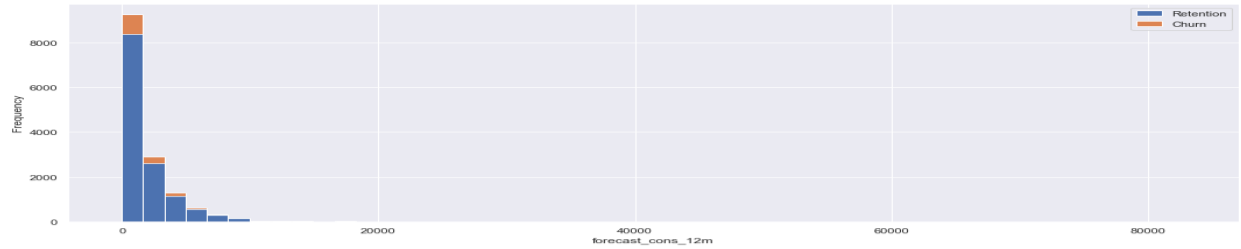

0
2000
4000
6000
8000
10000
12000
14000
imp_cons

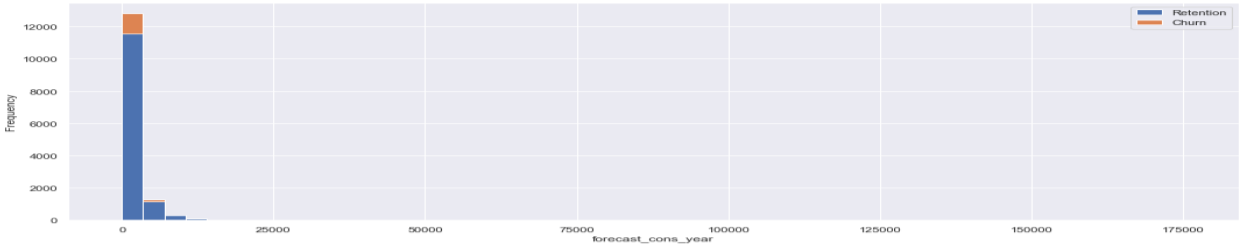
We will deal with skewness and outliers during feature engineering in the next exercise.

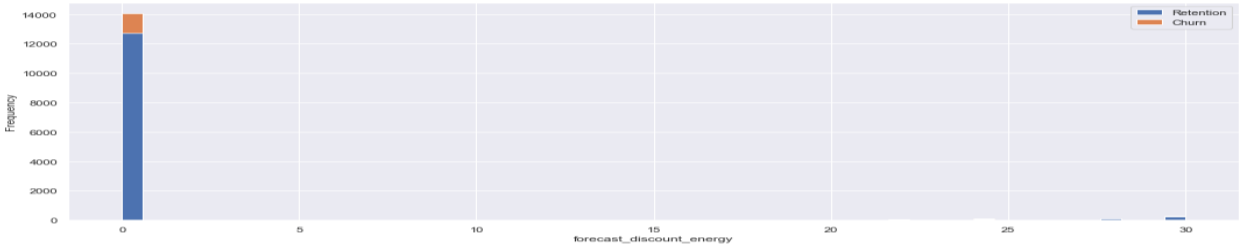
Forecast

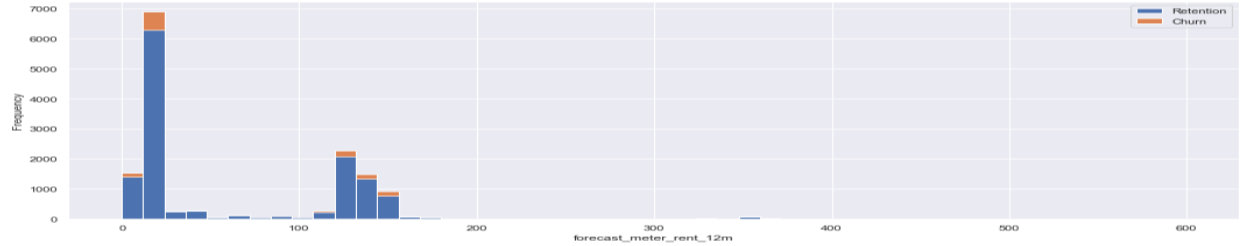
```
forecast = client_df[
    ["id", "forecast_cons_12m",

"forecast_cons_year","forecast_discount_energy","forecast_meter_rent_12m",
    "forecast_price_energy_off_peak","forecast_price_energy_peak",
    "forecast_price_pow_off_peak","churn"
    ]
]
```

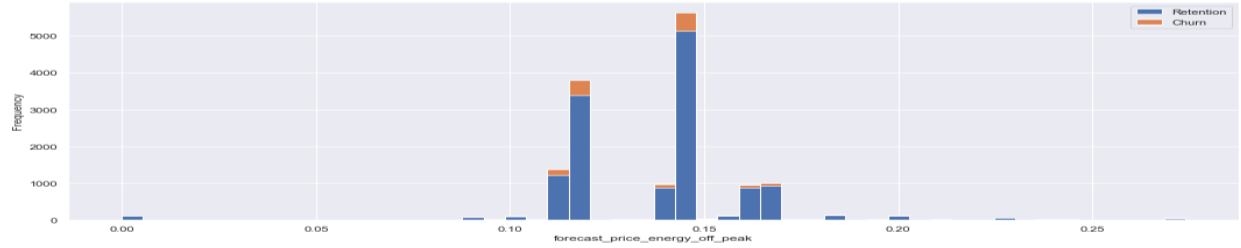
```
fig, axs = plt.subplots(nrows=7, figsize=(18,50))

# Plot histogram
plot_distribution(client_df, "forecast_cons_12m", axs[0])
plot_distribution(client_df, "forecast_cons_year", axs[1])
plot_distribution(client_df, "forecast_discount_energy", axs[2])
plot_distribution(client_df, "forecast_meter_rent_12m", axs[3])
plot_distribution(client_df, "forecast_price_energy_off_peak", axs[4])
plot_distribution(client_df, "forecast_price_energy_peak", axs[5])
plot_distribution(client_df, "forecast_price_pow_off_peak", axs[6])
```

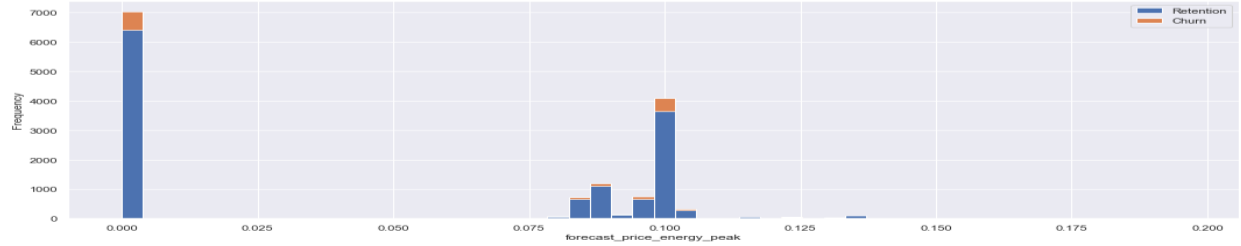

Retention
Churn
Frequency
0
2000
4000
6000
8000
0
20000
40000
60000
80000
forecast_cons_12m

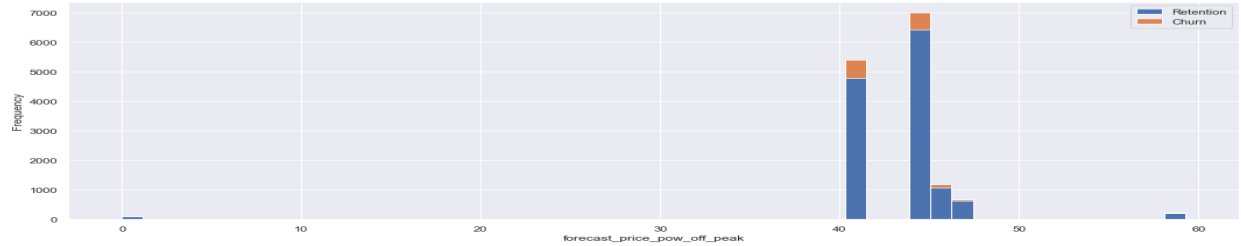

Retention
Churn
Frequency
0
2000
4000
6000
8000
10000
12000
0
25000
50000
75000
100000
125000
150000
175000
forecast_cons_year


Retention
Churn
Frequency
0
2000
4000
6000
8000
10000
12000
14000
0
5
10
15
20
25
30
forecast_discount_energy


Retention
Churn
Frequency
0
1000
2000
3000
4000
5000
6000
7000
0
100
200
300
400
500
600
forecast_meter_rent_12m


Retention
Churn
Frequency
0
1000
2000
3000
4000
5000
0.00
0.05
0.10
0.15
0.20
0.25
forecast_price_energy_off_peak

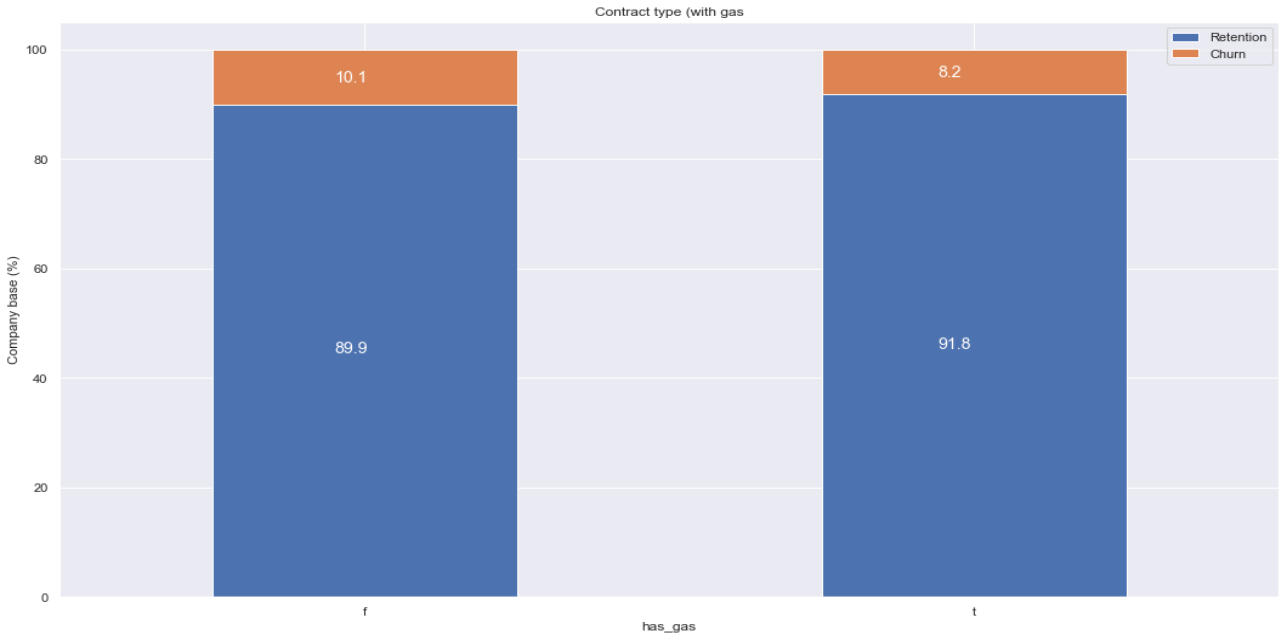

Retention
Churn
Frequency
0
1000
2000
3000
4000
5000
6000
7000
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200
forecast_price_energy_peak


Retention
Churn
Frequency
0
1000
2000
3000
4000
5000
6000
7000
0
10
20
30
40
50
60
forecast_price_pow_off_peak

Similarly to the consumption plots, we can observe that a lot of the variables are highly positively skewed, creating a very long tail for the higher values. We will make some transformations during the next exercise to correct for this skewness.

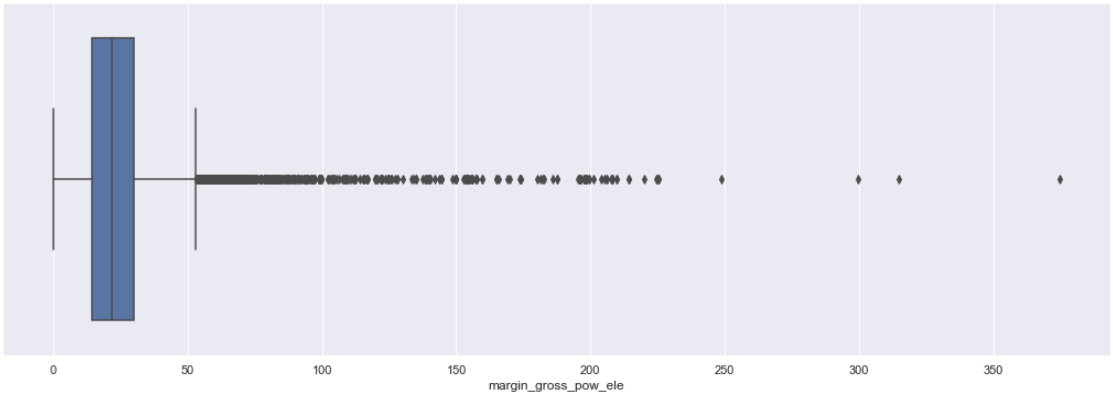
Contract type

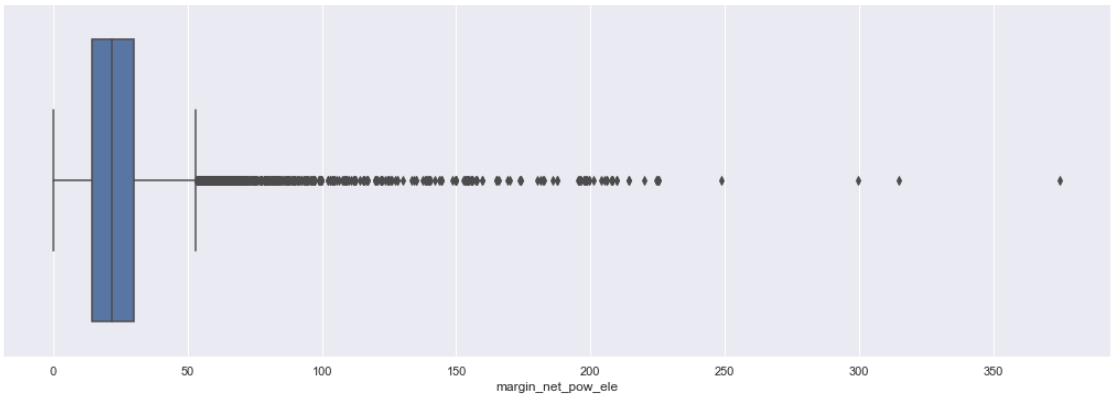
```
contract_type = client_df[['id', 'has_gas', 'churn']]
contract = contract_type.groupby([contract_type['churn'],
contract_type['has_gas']])['id'].count().unstack(level=0)
contract_percentage = (contract.div(contract.sum(axis=1), axis=0) *
100).sort_values(by=[1], ascending=False)

plot_stacked_bars(contract_percentage, 'Contract type (with gas')
```

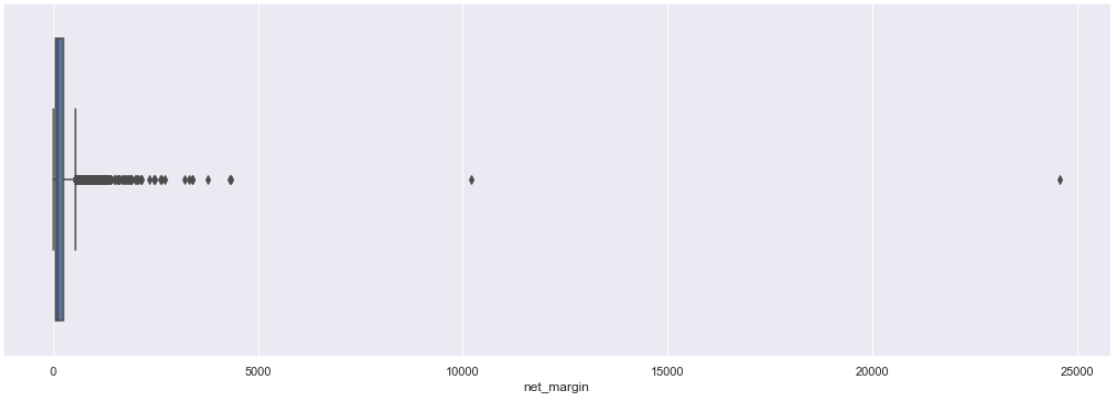


Margins

```
margin = client_df[['id', 'margin_gross_pow_ele', 'margin_net_pow_ele',
'net_margin']]

fig, axs = plt.subplots(nrows=3, figsize=(18,20))
# Plot histogram
sns.boxplot(margin["margin_gross_pow_ele"], ax=axs[0])
sns.boxplot(margin["margin_net_pow_ele"],ax=axs[1])
sns.boxplot(margin["net_margin"], ax=axs[2])
# Remove scientific notation
axs[0].ticklabel_format(style='plain', axis='x')
axs[1].ticklabel_format(style='plain', axis='x')
axs[2].ticklabel_format(style='plain', axis='x')
plt.show()
```



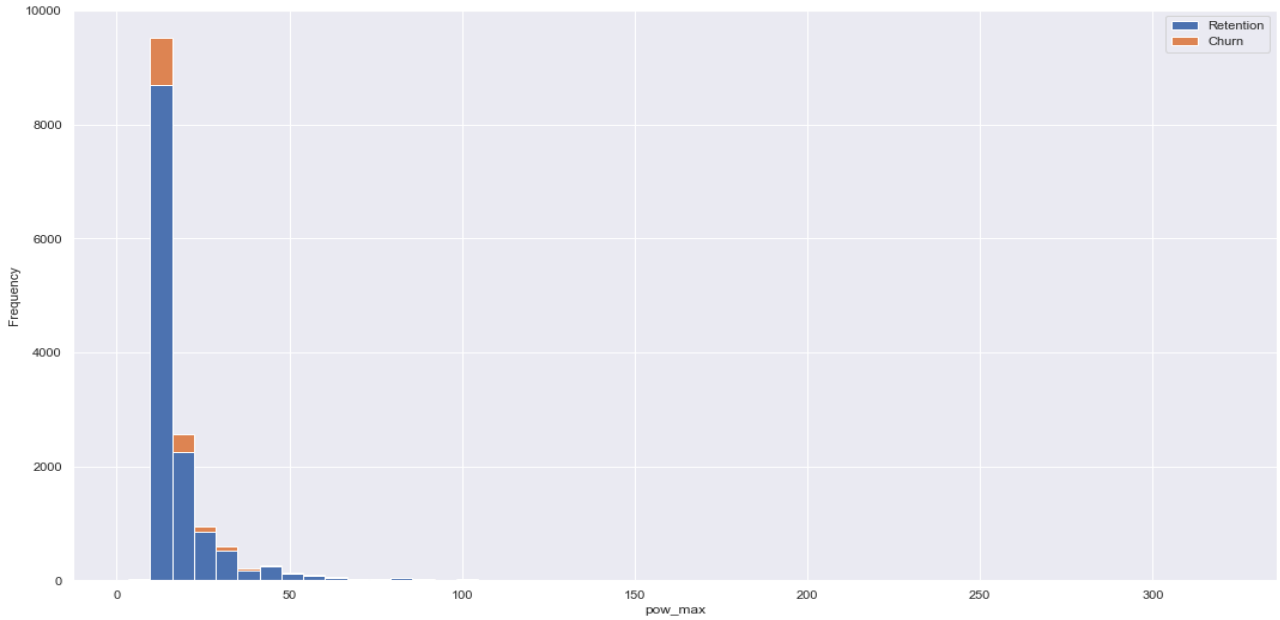




We can see some outliers here as well which we will deal with in the next exercise.

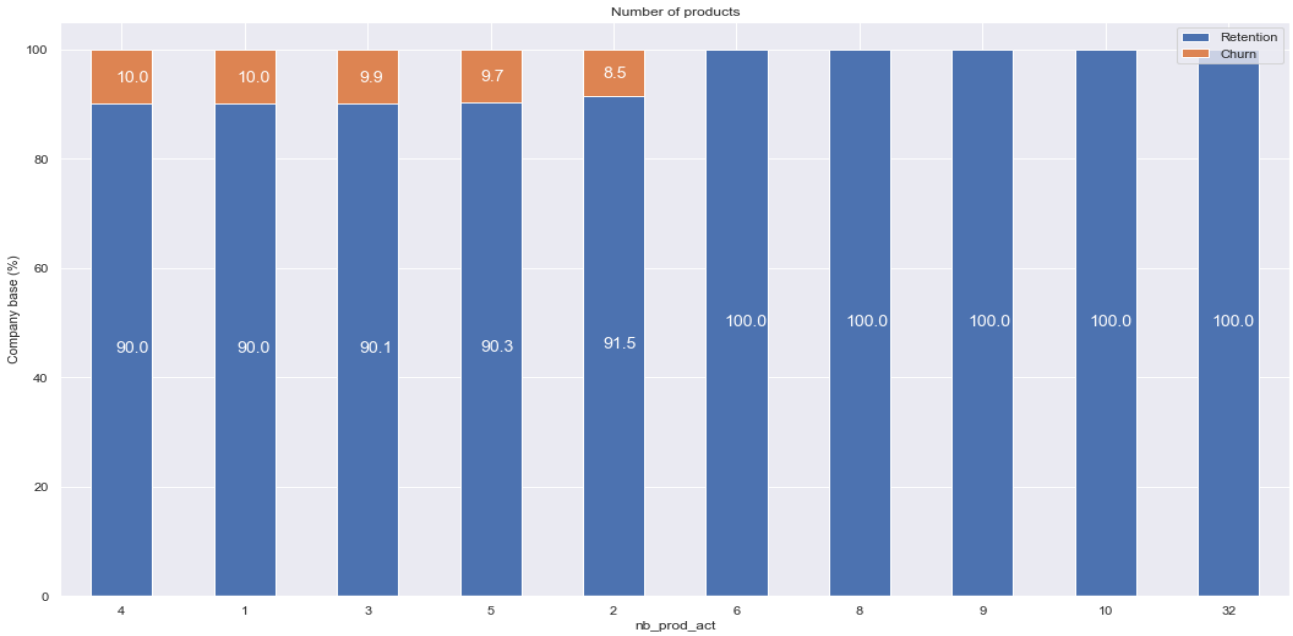
Subscribed power

```
power = client_df[['id', 'pow_max', 'churn']]

fig, axs = plt.subplots(nrows=1, figsize=(18, 10))
plot_distribution(power, 'pow_max', axs)
```

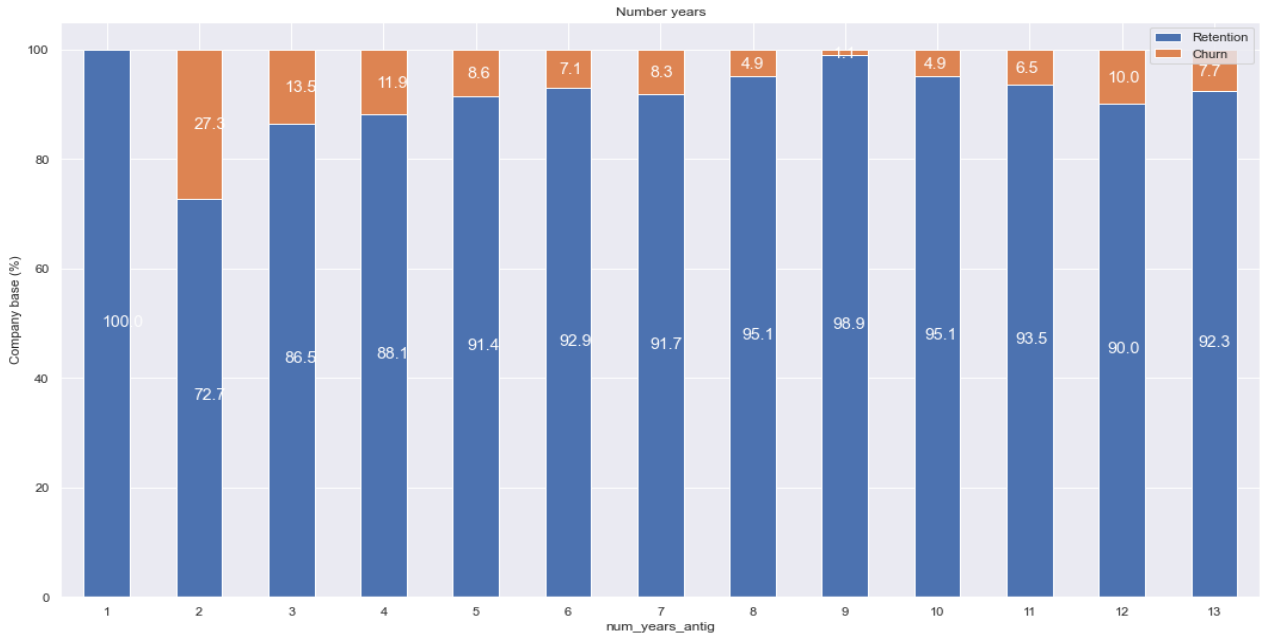


Other columns

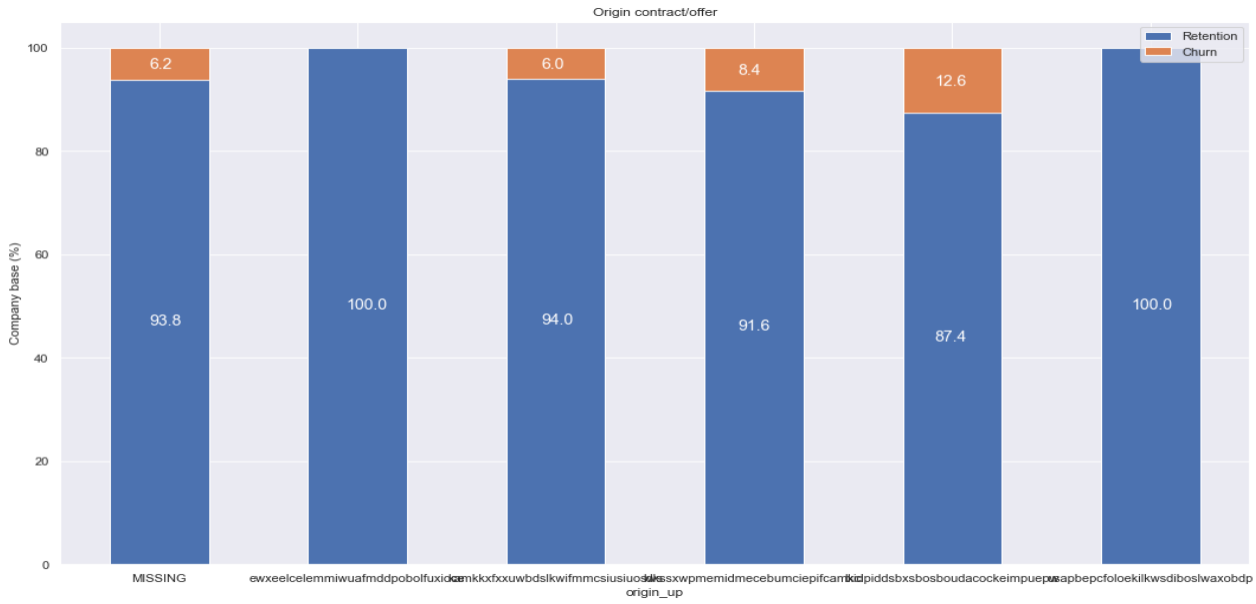
```
others = client_df[['id', 'nb_prod_act', 'num_years_antig', 'origin_up',
'churn']]
products =
others.groupby([others["nb_prod_act"],others["churn"]])["id"].count().unstack
(level=1)
products_percentage = (products.div(products.sum(axis=1),
axis=0)*100).sort_values(by=[1], ascending=False)

plot_stacked_bars(products_percentage, "Number of products")
```



```
years_antig =
others.groupby([others["num_years_antig"],others["churn"]])["id"].count().uns
tack(level=1)
years_antig_percentage = (years_antig.div(years_antig.sum(axis=1),
axis=0)*100)
plot_stacked_bars(years_antig_percentage, "Number years")
```




```
origin =
others.groupby([others["origin_up"],others["churn"]])["id"].count().unstack(l
evel=1)
origin_percentage = (origin.div(origin.sum(axis=1), axis=0)*100)
plot_stacked_bars(origin_percentage, "Origin contract/offer")
```



Feature Engineering and Modelling

---

1. Import packages
2. Load data
3. Feature engineering

---

1. Import packages

```
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

import pandas as pd
import numpy as np
import seaborn as sns
from datetime import datetime
import matplotlib.pyplot as plt

# Shows plots in jupyter notebook
%matplotlib inline

# Set plot style
sns.set(color_codes=True)
```

---

2. Load data

```
df = pd.read_csv('./clean_data_after_eda.csv')
df["date_activ"] = pd.to_datetime(df["date_activ"], format='%Y-%m-%d')
df["date_end"] = pd.to_datetime(df["date_end"], format='%Y-%m-%d')
df["date_modif_prod"] = pd.to_datetime(df["date_modif_prod"], format='%Y-%m-%
d')
df["date_renewal"] = pd.to_datetime(df["date_renewal"], format='%Y-%m-%d')

df.head(3)
```

```
                                 id                    channel_sales  \
0  24011ae4ebbe3035111d65fa7c15bc57  foosdfpfkusacimwkcsosbicdxkicaua
1  d29c2c54acc38ff3c0614d0a653813dd                           MISSING
2  764c75f661154dac3a6c254cd082ea7d  foosdfpfkusacimwkcsosbicdxkicaua

   cons_12m  cons_gas_12m  cons_last_month date_activ   date_end  \
0         0         54946                0 2013-06-15 2016-06-15
1      4660             0                0 2009-08-21 2016-08-30
2       544             0                0 2010-04-16 2016-04-16

  date_modif_prod date_renewal  forecast_cons_12m  ...  \
0      2015-11-01   2015-06-23               0.00  ...
```

```
1      2009-08-21   2015-08-31             189.95  ...
2      2010-04-16   2015-04-17              47.96  ...

   var_6m_price_off_peak_var  var_6m_price_peak_var  \
0                   0.000131          4.100838e-05
1                   0.000003          1.217891e-03
2                   0.000004          9.450150e-08

   var_6m_price_mid_peak_var  var_6m_price_off_peak_fix  \
0                   0.000908                   2.086294
1                   0.000000                   0.009482
2                   0.000000                   0.000000

   var_6m_price_peak_fix  var_6m_price_mid_peak_fix var_6m_price_off_peak  \
0              99.530517                  44.235794              2.086425
1               0.000000                   0.000000              0.009485
2               0.000000                   0.000000              0.000004

   var_6m_price_peak  var_6m_price_mid_peak  churn
0       9.953056e+01              44.236702      1
1       1.217891e-03               0.000000      0
2       9.450150e-08               0.000000      0

[3 rows x 44 columns]
```

---

3. Feature engineering

Difference between off-peak prices in December and preceding January

Below is the code created by your colleague to calculate the feature described above. Use this code to re-create this feature and then think about ways to build on this feature to create features with predictive power.

```
price_df = pd.read_csv('price_data.csv')
price_df["price_date"] = pd.to_datetime(price_df["price_date"], format='%Y-%m
-%d')
price_df.head()
```

```
                                 id price_date  price_off_peak_var  \
0  038af19179925da21a25619c5a24b745 2015-01-01            0.151367
1  038af19179925da21a25619c5a24b745 2015-02-01            0.151367
2  038af19179925da21a25619c5a24b745 2015-03-01            0.151367
3  038af19179925da21a25619c5a24b745 2015-04-01            0.149626
4  038af19179925da21a25619c5a24b745 2015-05-01            0.149626

   price_peak_var  price_mid_peak_var  price_off_peak_fix  price_peak_fix  \
0             0.0                 0.0           44.266931             0.0
1             0.0                 0.0           44.266931             0.0
```

```
2             0.0                 0.0           44.266931             0.0
3             0.0                 0.0           44.266931             0.0
4             0.0                 0.0           44.266931             0.0

   price_mid_peak_fix
0                 0.0
1                 0.0
2                 0.0
3                 0.0
4                 0.0
```

```
# Group off-peak prices by companies and month
monthly_price_by_id = price_df.groupby(['id', 'price_date']).agg({'price_off_
peak_var': 'mean', 'price_off_peak_fix': 'mean'}).reset_index()

# Get january and december prices
jan_prices = monthly_price_by_id.groupby('id').first().reset_index()
dec_prices = monthly_price_by_id.groupby('id').last().reset_index()

# Calculate the difference
diff = pd.merge(dec_prices.rename(columns={'price_off_peak_var': 'dec_1', 'pr
ice_off_peak_fix': 'dec_2'}), jan_prices.drop(columns='price_date'), on='id')
diff['offpeak_diff_dec_january_energy'] = diff['dec_1'] - diff['price_off_pea
k_var']
diff['offpeak_diff_dec_january_power'] = diff['dec_2'] - diff['price_off_peak
_fix']
diff = diff[['id', 'offpeak_diff_dec_january_energy','offpeak_diff_dec_januar
y_power']]
diff.head()
```

```
                                 id  offpeak_diff_dec_january_energy  \
0  0002203ffbb812588b632b9e628cc38d                        -0.006192
1  0004351ebdd665e6ee664792efc4fd13                        -0.004104
2  0010bcc39e42b3c2131ed2ce55246e3c                         0.050443
3  0010ee3855fdea87602a5b7aba8e42de                        -0.010018
4  00114d74e963e47177db89bc70108537                        -0.003994

   offpeak_diff_dec_january_power
0                        0.162916
1                        0.177779
2                        1.500000
3                        0.162916
4                       -0.000001
```

```
df = pd.merge(df, diff, on='id')
df.head()
```

```
                                 id                      channel_sales  \
0  24011ae4ebbe3035111d65fa7c15bc57  foosdfpfkusacimwkcsosbicdxkicaua
1  d29c2c54acc38ff3c0614d0a653813dd                           MISSING
```

```
2  764c75f661154dac3a6c254cd082ea7d  foosdfpfkusacimwkcsosbicdxkicaua
3  bba03439a292a1e166f80264c16191cb  lmkebamcaaclubfxadlmueccxoimlema
4  149d57cf92fc41cf94415803a877cb4b                           MISSING

   cons_12m  cons_gas_12m  cons_last_month date_activ   date_end  \
0         0         54946                0 2013-06-15 2016-06-15
1      4660             0                0 2009-08-21 2016-08-30
2       544             0                0 2010-04-16 2016-04-16
3      1584             0                0 2010-03-30 2016-03-30
4      4425             0              526 2010-01-13 2016-03-07

  date_modif_prod date_renewal  forecast_cons_12m  ...  \
0      2015-11-01   2015-06-23               0.00  ...
1      2009-08-21   2015-08-31             189.95  ...
2      2010-04-16   2015-04-17              47.96  ...
3      2010-03-30   2015-03-31             240.04  ...
4      2010-01-13   2015-03-09             445.75  ...

   var_6m_price_mid_peak_var  var_6m_price_off_peak_fix  \
0               9.084737e-04                   2.086294
1               0.000000e+00                   0.009482
2               0.000000e+00                   0.000000
3               0.000000e+00                   0.000000
4               4.860000e-10                   0.000000

   var_6m_price_peak_fix  var_6m_price_mid_peak_fix  var_6m_price_off_peak  \
0              99.530517                  44.235794               2.086425
1               0.000000                   0.000000               0.009485
2               0.000000                   0.000000               0.000004
3               0.000000                   0.000000               0.000003
4               0.000000                   0.000000               0.000011

   var_6m_price_peak var_6m_price_mid_peak  churn  \
0       9.953056e+01          4.423670e+01      1
1       1.217891e-03          0.000000e+00      0
2       9.450150e-08          0.000000e+00      0
3       0.000000e+00          0.000000e+00      0
4       2.896760e-06          4.860000e-10      0

   offpeak_diff_dec_january_energy  offpeak_diff_dec_january_power
0                         0.020057                       3.700961
1                        -0.003767                       0.177779
2                        -0.004670                       0.177779
3                        -0.004547                       0.177779
4                        -0.006192                       0.162916

[5 rows x 46 columns]
```

Average price changes across periods

We can now enhance the feature that our colleague made by calculating the average price changes across individual periods, instead of the entire year.

```
# Aggregate average prices per period by company
mean_prices = price_df.groupby(['id']).agg({
    'price_off_peak_var': 'mean',
    'price_peak_var': 'mean',
    'price_mid_peak_var': 'mean',
    'price_off_peak_fix': 'mean',
    'price_peak_fix': 'mean',
    'price_mid_peak_fix': 'mean'
}).reset_index()

# Calculate the mean difference between consecutive periods
mean_prices['off_peak_peak_var_mean_diff'] = mean_prices['price_off_peak_var'
] - mean_prices['price_peak_var']
mean_prices['peak_mid_peak_var_mean_diff'] = mean_prices['price_peak_var'] -
mean_prices['price_mid_peak_var']
mean_prices['off_peak_mid_peak_var_mean_diff'] = mean_prices['price_off_peak_
var'] - mean_prices['price_mid_peak_var']
mean_prices['off_peak_peak_fix_mean_diff'] = mean_prices['price_off_peak_fix'
] - mean_prices['price_peak_fix']
mean_prices['peak_mid_peak_fix_mean_diff'] = mean_prices['price_peak_fix'] -
mean_prices['price_mid_peak_fix']
mean_prices['off_peak_mid_peak_fix_mean_diff'] = mean_prices['price_off_peak_
fix'] - mean_prices['price_mid_peak_fix']

columns = [
    'id',
    'off_peak_peak_var_mean_diff',
    'peak_mid_peak_var_mean_diff',
    'off_peak_mid_peak_var_mean_diff',
    'off_peak_peak_fix_mean_diff',
    'peak_mid_peak_fix_mean_diff',
    'off_peak_mid_peak_fix_mean_diff'
]
df = pd.merge(df, mean_prices[columns], on='id')
df.head()
```

```
                                 id                      channel_sales  \
0  24011ae4ebbe3035111d65fa7c15bc57  foosdfpfkusacimwkcsosbicdxkicaua
1  d29c2c54acc38ff3c0614d0a653813dd                           MISSING
2  764c75f661154dac3a6c254cd082ea7d  foosdfpfkusacimwkcsosbicdxkicaua
3  bba03439a292a1e166f80264c16191cb  lmkebamcaaclubfxadlmueccxoimlema
4  149d57cf92fc41cf94415803a877cb4b                           MISSING

   cons_12m  cons_gas_12m  cons_last_month date_activ   date_end  \
0         0         54946                0 2013-06-15 2016-06-15
```

```
1      4660             0                0 2009-08-21 2016-08-30
2       544             0                0 2010-04-16 2016-04-16
3      1584             0                0 2010-03-30 2016-03-30
4      4425             0              526 2010-01-13 2016-03-07

  date_modif_prod date_renewal  forecast_cons_12m  ...  var_6m_price_mid_peak
\
0      2015-11-01   2015-06-23               0.00  ...           4.423670e+01
1      2009-08-21   2015-08-31             189.95  ...           0.000000e+00
2      2010-04-16   2015-04-17              47.96  ...           0.000000e+00
3      2010-03-30   2015-03-31             240.04  ...           0.000000e+00
4      2010-01-13   2015-03-09             445.75  ...           4.860000e-10

   churn  offpeak_diff_dec_january_energy  offpeak_diff_dec_january_power  \
0      1                         0.020057                        3.700961
1      0                        -0.003767                        0.177779
2      0                        -0.004670                        0.177779
3      0                        -0.004547                        0.177779
4      0                        -0.006192                        0.162916

   off_peak_peak_var_mean_diff  peak_mid_peak_var_mean_diff  \
0                     0.024038                     0.034219
1                     0.142485                     0.007124
2                     0.082090                     0.088421
3                     0.151210                     0.000000
4                     0.020536                     0.030773

  off_peak_mid_peak_var_mean_diff  off_peak_peak_fix_mean_diff  \
0                        0.058257                    18.590255
1                        0.149609                    44.311375
2                        0.170512                    44.385450
3                        0.151210                    44.400265
4                        0.051309                    16.275263

   peak_mid_peak_fix_mean_diff  off_peak_mid_peak_fix_mean_diff
0                     7.450670                        26.040925
1                     0.000000                        44.311375
2                     0.000000                        44.385450
3                     0.000000                        44.400265
4                     8.137629                        24.412893

[5 rows x 52 columns]
```

This feature may be useful because it adds more granularity to the existing feature that our colleague found to be useful. Instead of looking at differences across an entire year, we have now created features that look at mean average price differences across different time periods (`off_peak`, `peak`, `mid_peak`). The dec-jan feature may reveal macro patterns that occur over an entire year, whereas inter-time-period features may reveal patterns on a micro scale between months.

Max price changes across periods and months

Another way we can enhance the feature from our colleague is to look at the maximum change in prices across periods and months.

```
# Aggregate average prices per period by company
mean_prices_by_month = price_df.groupby(['id', 'price_date']).agg({
    'price_off_peak_var': 'mean',
    'price_peak_var': 'mean',
    'price_mid_peak_var': 'mean',
    'price_off_peak_fix': 'mean',
    'price_peak_fix': 'mean',
    'price_mid_peak_fix': 'mean'
}).reset_index()

# Calculate the mean difference between consecutive periods
mean_prices_by_month['off_peak_peak_var_mean_diff'] = mean_prices_by_month['p
rice_off_peak_var'] - mean_prices_by_month['price_peak_var']
mean_prices_by_month['peak_mid_peak_var_mean_diff'] = mean_prices_by_month['p
rice_peak_var'] - mean_prices_by_month['price_mid_peak_var']
mean_prices_by_month['off_peak_mid_peak_var_mean_diff'] = mean_prices_by_mont
h['price_off_peak_var'] - mean_prices_by_month['price_mid_peak_var']
mean_prices_by_month['off_peak_peak_fix_mean_diff'] = mean_prices_by_month['p
rice_off_peak_fix'] - mean_prices_by_month['price_peak_fix']
mean_prices_by_month['peak_mid_peak_fix_mean_diff'] = mean_prices_by_month['p
rice_peak_fix'] - mean_prices_by_month['price_mid_peak_fix']
mean_prices_by_month['off_peak_mid_peak_fix_mean_diff'] = mean_prices_by_mont
h['price_off_peak_fix'] - mean_prices_by_month['price_mid_peak_fix']

# Calculate the maximum monthly difference across time periods
max_diff_across_periods_months = mean_prices_by_month.groupby(['id']).agg({
    'off_peak_peak_var_mean_diff': 'max',
    'peak_mid_peak_var_mean_diff': 'max',
    'off_peak_mid_peak_var_mean_diff': 'max',
    'off_peak_peak_fix_mean_diff': 'max',
    'peak_mid_peak_fix_mean_diff': 'max',
    'off_peak_mid_peak_fix_mean_diff': 'max'
}).reset_index().rename(
    columns={
        'off_peak_peak_var_mean_diff': 'off_peak_peak_var_max_monthly_diff',
        'peak_mid_peak_var_mean_diff': 'peak_mid_peak_var_max_monthly_diff',
        'off_peak_mid_peak_var_mean_diff': 'off_peak_mid_peak_var_max_monthly
_diff',
        'off_peak_peak_fix_mean_diff': 'off_peak_peak_fix_max_monthly_diff',
        'peak_mid_peak_fix_mean_diff': 'peak_mid_peak_fix_max_monthly_diff',
        'off_peak_mid_peak_fix_mean_diff': 'off_peak_mid_peak_fix_max_monthly
_diff'
```

```
    }
)

columns = [
    'id',
    'off_peak_peak_var_max_monthly_diff',
    'peak_mid_peak_var_max_monthly_diff',
    'off_peak_mid_peak_var_max_monthly_diff',
    'off_peak_peak_fix_max_monthly_diff',
    'peak_mid_peak_fix_max_monthly_diff',
    'off_peak_mid_peak_fix_max_monthly_diff'
]

df = pd.merge(df, max_diff_across_periods_months[columns], on='id')
df.head()
```

	id	channel_sales	
0	24011ae4ebbe3035111d65fa7c15bc57	foosdfpfkusacimwkcsosbicdxkicaua	
1	d29c2c54acc38ff3c0614d0a653813dd	MISSING	
2	764c75f661154dac3a6c254cd082ea7d	foosdfpfkusacimwkcsosbicdxkicaua	
3	bba03439a292a1e166f80264c16191cb	lmkebamcaaclubfxadlmueccxoimlema	
4	149d57cf92fc41cf94415803a877cb4b	MISSING	

	cons_12m	cons_gas_12m	cons_last_month	date_activ	date_end	\
0	0	54946	0	2013-06-15	2016-06-15	
1	4660	0	0	2009-08-21	2016-08-30	
2	544	0	0	2010-04-16	2016-04-16	
3	1584	0	0	2010-03-30	2016-03-30	
4	4425	0	526	2010-01-13	2016-03-07	

	date_modif_prod	date_renewal	forecast_cons_12m	...	\
0	2015-11-01	2015-06-23	0.00	...	
1	2009-08-21	2015-08-31	189.95	...	
2	2010-04-16	2015-04-17	47.96	...	
3	2010-03-30	2015-03-31	240.04	...	
4	2010-01-13	2015-03-09	445.75	...	

	off_peak_mid_peak_var_mean_diff	off_peak_peak_fix_mean_diff	\
0	0.058257	18.590255	
1	0.149609	44.311375	
2	0.170512	44.385450	
3	0.151210	44.400265	
4	0.051309	16.275263	

	peak_mid_peak_fix_mean_diff	off_peak_mid_peak_fix_mean_diff	\
0	7.450670	26.040925	
1	0.000000	44.311375	
2	0.000000	44.385450	
3	0.000000	44.400265	
4	8.137629	24.412893	

```
   off_peak_peak_var_max_monthly_diff  peak_mid_peak_var_max_monthly_diff  \
0                            0.060550                           0.085483
1                            0.151367                           0.085483
2                            0.084587                           0.089162
3                            0.153133                           0.000000
4                            0.022225                           0.033743

  off_peak_mid_peak_var_max_monthly_diff  off_peak_peak_fix_max_monthly_diff
\
0                               0.146033                          44.266930
1                               0.151367                          44.444710
2                               0.172468                          44.444710
3                               0.153133                          44.444710
4                               0.055866                          16.291555

   peak_mid_peak_fix_max_monthly_diff  off_peak_mid_peak_fix_max_monthly_diff
0                            8.145775                                44.26693
1                            0.000000                                44.44471
2                            0.000000                                44.44471
3                            0.000000                                44.44471
4                            8.145775                                24.43733

[5 rows x 58 columns]
```

Calculating the maximum price change between months and time periods could be a good feature to create because customers might be more likely to churn if they experience sudden large price changes.

(BONUS) Further feature engineering

This section covers extra feature engineering that you may have thought of, as well as different ways you can transform your data to account for some of its statistical properties that we saw before, such as skewness.

Tenure

How long a company has been a client of PowerCo.

```
df['date_activ'] = pd.to_datetime(df['date_activ'])
df['date_end']   = pd.to_datetime(df['date_end'])

df['tenure'] = ((df['date_end'] - df['date_activ']).dt.days// 365)

df.groupby(['tenure']).agg({'churn': 'mean'}).sort_values(by='tenure', ascend
ing=False)
```

```
            churn
tenure
13      0.095238
12      0.083333
11      0.059783
10      0.045455
9       0.012500
8       0.047244
7       0.075472
6       0.075407
5       0.091999
4       0.127473
3       0.143874
2       0.176471
```

We can see that companies who have only been a client for 4 or less months are much more likely to churn compared to companies that have been a client for longer. Interestingly, the difference between 4 and 5 months is about 4%, which represents a large jump in likelihood for a customer to churn compared to the other differences between ordered tenure values. Perhaps this reveals that getting a customer to over 4 months tenure is actually a large milestone with respect to keeping them as a long term customer.

This is an interesting feature to keep for modelling because tenure is clearly correlated with churn probability.

Transforming dates into months

- months_activ = Number of months active until reference date (Jan 2016)
- months_to_end = Number of months of the contract left until reference date (Jan 2016)
- months_modif_prod = Number of months since last modification until reference date (Jan 2016)
- months_renewal = Number of months since last renewal until reference date (Jan 2016)

```
def convert_months(reference_date, df, column):
    """
    return number of whole months between two dates
    """
    dates = pd.to_datetime(df[column])

    year_diff  = reference_date.year  - dates.dt.year
    month_diff = reference_date.month - dates.dt.month

    # initial month count
    months = year_diff * 12 + month_diff

    # subtract 1 if the reference day hasn't been reached yet in that month
    months -= (reference_date.day < dates.dt.day).astype(int)

    return months
```

Dates as a datetime object are not useful for a predictive model, so we needed to use the datetimes to create some other features that may hold some predictive power.

Using intuition, one could assume that a client who has been an active client of PowerCo for a longer amount of time may have more loyalty to the brand and is more likely to stay, whereas a newer client may be more volatile. Hence the addition of the `months_activ` feature.

In addition, if customers are coming up to a contract end date, there may be a natural break point where it is cheaper or involves less friction to change. Therefore, `months_to_end` could be an interesting feature.

Alternatively, contract modification or renewal is a sign of engagement and may indicate clients are less likely to churn, so we can add two features to capture this: `months_modif_prod` and `months_renewal`.

```
# Create reference date
reference_date = datetime(2016, 1, 1)

# Create columns
df['months_activ'] = convert_months(reference_date, df, 'date_activ')
df['months_to_end'] = -convert_months(reference_date, df, 'date_end')
df['months_modif_prod'] = convert_months(reference_date, df, 'date_modif_prod
')
df['months_renewal'] = convert_months(reference_date, df, 'date_renewal')

# We no longer need the datetime columns that we used for feature engineering
, so we can drop them
remove = [
    'date_activ',
    'date_end',
    'date_modif_prod',
    'date_renewal'
]

df = df.drop(columns=remove)
df.head()
```

```
                                 id                    channel_sales  \
0  24011ae4ebbe3035111d65fa7c15bc57  foosdfpfkusacimwkcsosbicdxkicaua
1  d29c2c54acc38ff3c0614d0a653813dd                           MISSING
2  764c75f661154dac3a6c254cd082ea7d  foosdfpfkusacimwkcsosbicdxkicaua
3  bba03439a292a1e166f80264c16191cb  lmkebamcaaclubfxadlmueccxoimlema
4  149d57cf92fc41cf94415803a877cb4b                           MISSING

   cons_12m  cons_gas_12m  cons_last_month  forecast_cons_12m  \
0         0         54946                0               0.00
1      4660             0                0             189.95
2       544             0                0              47.96
3      1584             0                0             240.04
```

```
4      4425             0              526            445.75

   forecast_cons_year  forecast_discount_energy  forecast_meter_rent_12m  \
0                   0                       0.0                     1.78
1                   0                       0.0                    16.27
2                   0                       0.0                    38.72
3                   0                       0.0                    19.83
4                 526                       0.0                   131.73

   forecast_price_energy_off_peak  ...  peak_mid_peak_var_max_monthly_diff  \
0                        0.114481  ...                            0.085483
1                        0.145711  ...                            0.085483
2                        0.165794  ...                            0.089162
3                        0.146694  ...                            0.000000
4                        0.116900  ...                            0.033743

   off_peak_mid_peak_var_max_monthly_diff off_peak_peak_fix_max_monthly_diff
\
0                                0.146033                           44.266930
1                                0.151367                           44.444710
2                                0.172468                           44.444710
3                                0.153133                           44.444710
4                                0.055866                           16.291555

   peak_mid_peak_fix_max_monthly_diff  off_peak_mid_peak_fix_max_monthly_diff
\
0                            8.145775                                44.26693
1                            0.000000                                44.44471
2                            0.000000                                44.44471
3                            0.000000                                44.44471
4                            8.145775                                24.43733

   tenure  months_activ  months_to_end  months_modif_prod months_renewal
0       3            30              6                  2              6
1       7            76              8                 76              4
2       6            68              4                 68              8
3       6            69              3                 69              9
4       6            71              3                 71              9

[5 rows x 59 columns]
```

Transforming Boolean data

has_gas

We simply want to transform this column from being categorical to being a binary flag

```
df['has_gas'] = df['has_gas'].replace(['t', 'f'], [1, 0])
df.groupby(['has_gas']).agg({'churn': 'mean'})
```

```
            churn
has_gas
0        0.100544
1        0.081856
```

Customers who buy multiple products from PowerCo may have higher switching costs, and therefore could be more loyal. Given that customers who buy gas are 2% less likely to churn, this appears to be an important feature.

Transforming categorical data

A predictive model cannot accept categorical or `string` values, so these variables need to be transformed before they can be used in modeling. One method is to map each category to an integer (label encoding), however this is not always appropriate because it then introduces the concept of an order into a feature which may not inherently be present `0 < 1 < 2 < 3 ...`

Another way to encode categorical features is to use `dummy variables` (also know as `one hot encoding`). This create a new feature for every unique value of a categorical column, and fills this column with either a 1 or a 0 to indicate that this company does or does not belong to this category. Since it isn't useful to add features to our model where there are 0's for most rows, we will drop any category that shows up less than 100 times in our dataset.

channel_sales and origin_up

```
cols = ['channel_sales', 'origin_up']
min_count = 100

# get counts per category so we can filter out rare categories
value_counts = {col: df[col].value_counts() for col in cols}

df = pd.get_dummies(df, columns=cols, prefix=cols, dtype=int)

# drop rare columns
for col in cols:
    keep_cols = set(value_counts[col][value_counts[col] >= min_count].index.a
stype(str))

    dummy_cols = [c for c in df.columns if c.startswith(f"{col}_")]
    drop_cols = [c for c in dummy_cols if c.split(f"{col}_", 1)[1] not in kee
p_cols]
    df.drop(columns=drop_cols, inplace=True)
```

Transforming numerical data

In the previous exercise we saw that some variables were highly skewed. The reason why we need to treat skewness is because some predictive models have inherent assumptions about the distribution of the features. Such models are called `parametric` models, and they typically assume that all variables are both independent and normally distributed.

Skewness isn't always a bad thing, but as a rule of thumb it is good practice to treat highly skewed variables to make them more normally distributed. This can also increase the speed at which predictive models are able to converge to the best solution.

There are many ways that you can treat skewed variables. You can apply transformations such as:

- Square root
- Cubic root
- Logarithm

to a continuous numeric column and you will notice the distribution changes. For this use case we will use the 'Logarithm' transformation for the positively skewed features.

Note: We cannot apply log to a value of 0, so we will add a constant of 1 to all the values

We will check the distributions before and after transformation, and expect the standard deviations to decrease.

```
skewed_cols = [
    'cons_12m',
    'cons_gas_12m',
    'cons_last_month',
    'forecast_cons_12m',
    'forecast_cons_year',
    'forecast_discount_energy',
    'forecast_meter_rent_12m',
    'forecast_price_energy_off_peak',
    'forecast_price_energy_peak',
    'forecast_price_pow_off_peak'
]

print("Before log transform:")
print(df[skewed_cols].describe())

df[skewed_cols] = np.log10(df[skewed_cols] + 1)

print("After log transform:")
print(df[skewed_cols].describe())
```

```
Before log transform:
           cons_12m  cons_gas_12m  cons_last_month  forecast_cons_12m  \
count  1.460600e+04  1.460600e+04     14606.000000       14606.000000
mean   1.592203e+05  2.809238e+04     16090.269752        1868.614880
std    5.734653e+05  1.629731e+05     64364.196422        2387.571531
min    0.000000e+00  0.000000e+00         0.000000           0.000000
25%    5.674750e+03  0.000000e+00         0.000000         494.995000
50%    1.411550e+04  0.000000e+00       792.500000        1112.875000
75%    4.076375e+04  0.000000e+00      3383.000000        2401.790000
max    6.207104e+06  4.154590e+06    771203.000000       82902.830000
```

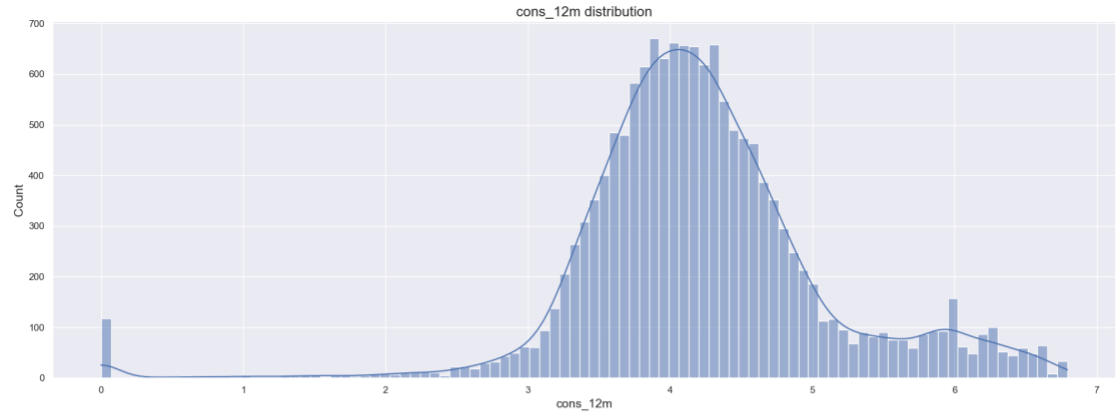
```
       forecast_cons_year  forecast_discount_energy  forecast_meter_rent_12m
\
count        14606.000000              14606.000000             14606.000000
mean          1399.762906                  0.966726                63.086871
std           3247.786255                  5.108289                66.165783
min              0.000000                  0.000000                 0.000000
25%              0.000000                  0.000000                16.180000
50%            314.000000                  0.000000                18.795000
75%           1745.750000                  0.000000               131.030000
max         175375.000000                 30.000000               599.310000

       forecast_price_energy_off_peak  forecast_price_energy_peak  \
count                    14606.000000                14606.000000
mean                         0.137283                    0.050491
std                          0.024623                    0.049037
min                          0.000000                    0.000000
25%                          0.116340                    0.000000
50%                          0.143166                    0.084138
75%                          0.146348                    0.098837
max                          0.273963                    0.195975

       forecast_price_pow_off_peak
count                 14606.000000
mean                     43.130056
std                       4.485988
min                       0.000000
25%                      40.606701
50%                      44.311378
75%                      44.311378
max                      59.266378
After log transform:
           cons_12m  cons_gas_12m  cons_last_month  forecast_cons_12m  \
count  14606.000000  14606.000000     14606.000000       14606.000000
mean       4.223939      0.779244         2.264646           2.962177
std        0.884515      1.717071         1.769305           0.683592
min        0.000000      0.000000         0.000000           0.000000
25%        3.754023      0.000000         0.000000           2.695477
50%        4.149727      0.000000         2.899547           3.046836
75%        4.610285      0.000000         3.529430           3.380716
max        6.792889      6.618528         5.887169           4.918575

       forecast_cons_year  forecast_discount_energy  forecast_meter_rent_12m
\
count        14606.000000              14606.000000             14606.000000
mean             1.784610                  0.050918                 1.517203
std              1.584986                  0.267388                 0.571481
min              0.000000                  0.000000                 0.000000
25%              0.000000                  0.000000                 1.235023
```

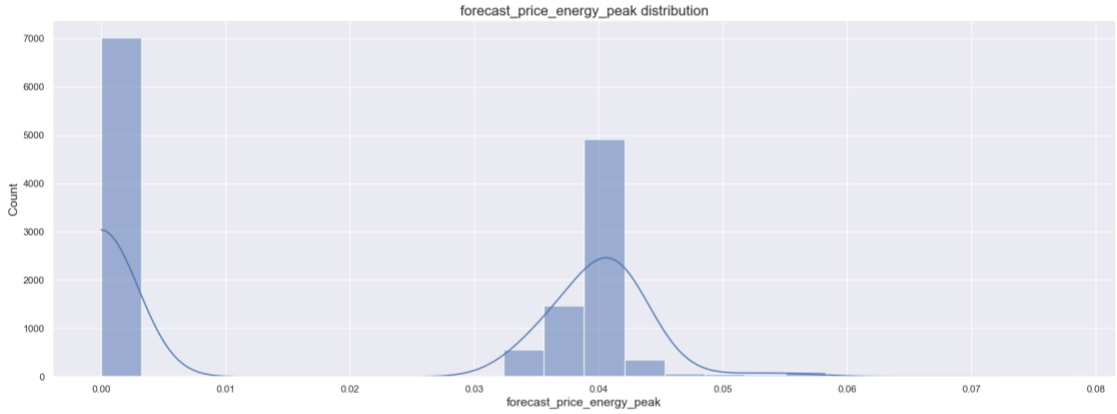
```
50%              2.498311                  0.000000                  1.296555
75%              3.242231                  0.000000                  2.120673
max              5.243970                  1.491362                  2.778376

       forecast_price_energy_off_peak  forecast_price_energy_peak  \
count                    14606.000000                14606.000000
mean                         0.055766                    0.020918
std                          0.009438                    0.020296
min                          0.000000                    0.000000
25%                          0.047796                    0.000000
50%                          0.058109                    0.035085
75%                          0.059316                    0.040933
max                          0.105157                    0.077722

       forecast_price_pow_off_peak
count                 14606.000000
mean                      1.636058
std                       0.134237
min                       0.000000
25%                       1.619163
50%                       1.656207
75%                       1.656207
max                       1.780075
```

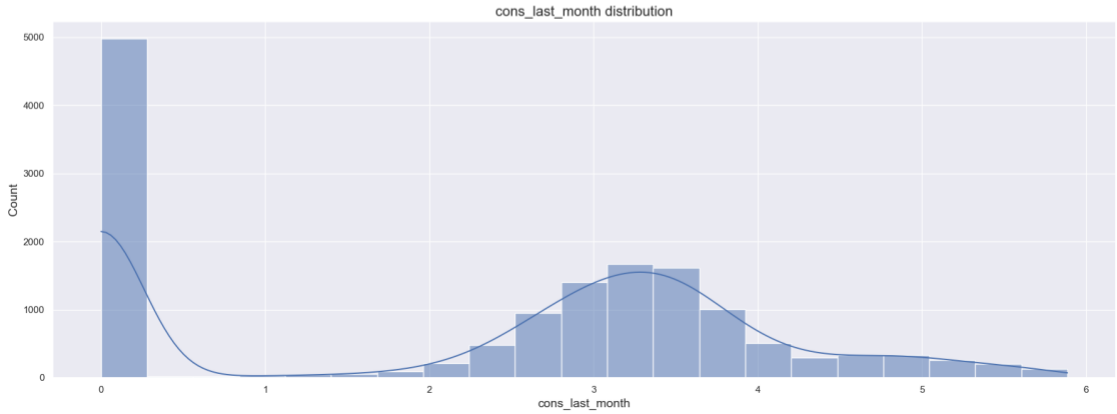
Now we can see that for the majority of the features, the standard deviation is much lower after transformation. This is a good thing as it shows that these features are more stable and predictable now.

Let's plot a few of these distributions too.

```
cols = ["cons_12m", "forecast_price_energy_peak", "cons_last_month"]

# Set up subplots
fig, axs = plt.subplots(nrows=len(cols), figsize=(18, 20))

# Loop through each column and plot
for ax, col in zip(axs, cols):
    sns.histplot(df[col].dropna(), kde=True, ax=ax)
    ax.set_title(f"{col} distribution", fontsize=16)
    ax.set_xlabel(col, fontsize=14)
    ax.set_ylabel("Count", fontsize=14)

plt.tight_layout()
plt.show()
```








```
df.head()
```

	id	cons_12m	cons_gas_12m	cons_last_month
0	24011ae4ebbe3035111d65fa7c15bc57	0.000000	4.739944	0.000000
1	d29c2c54acc38ff3c0614d0a653813dd	3.668479	0.000000	0.000000
2	764c75f661154dac3a6c254cd082ea7d	2.736397	0.000000	0.000000
3	bba03439a292a1e166f80264c16191cb	3.200029	0.000000	0.000000
4	149d57cf92fc41cf94415803a877cb4b	3.646011	0.000000	2.721811

	forecast_cons_12m	forecast_cons_year	forecast_discount_energy
0	0.000000	0.000000	0.0
1	2.280920	0.000000	0.0

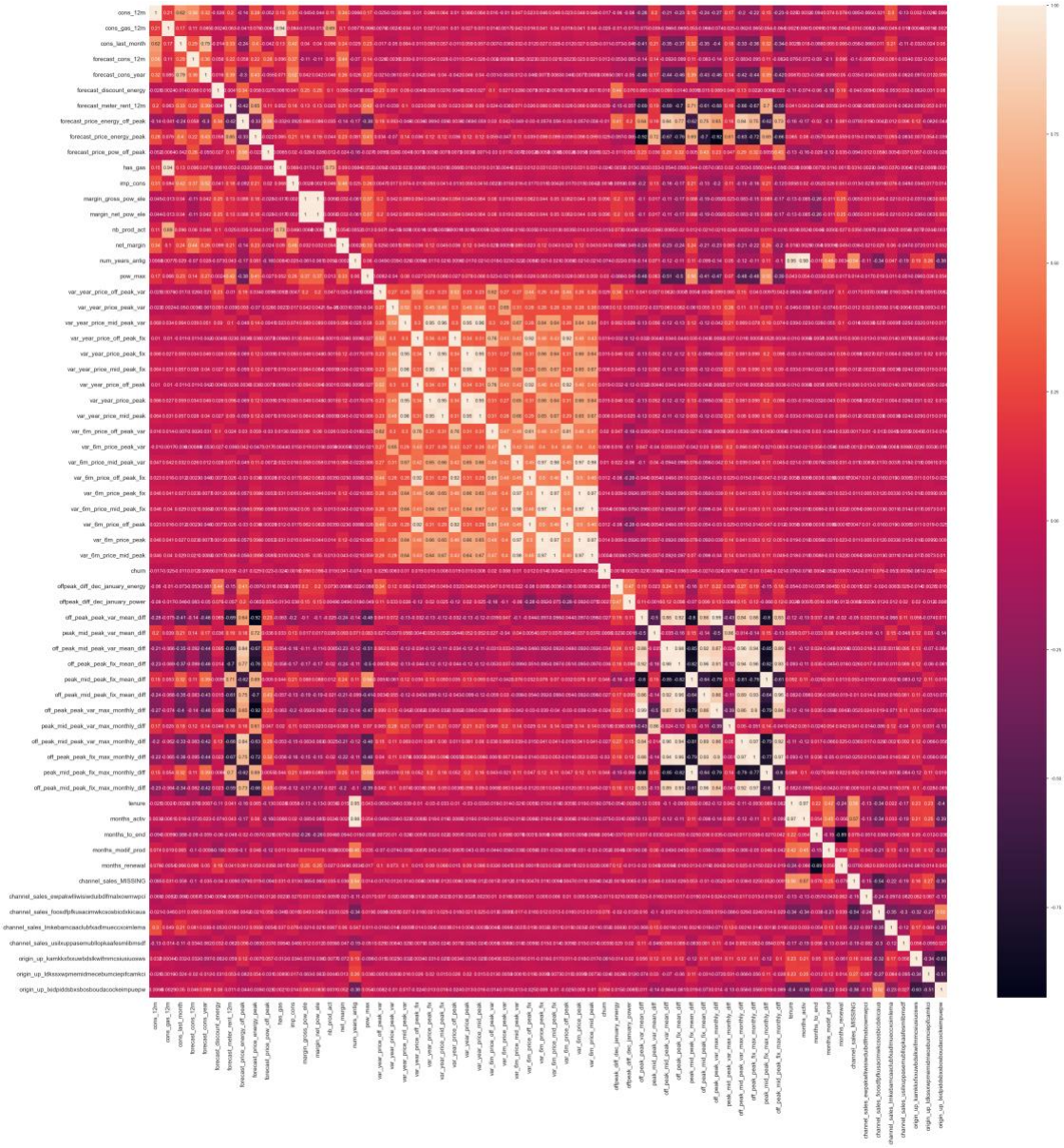
```
2          1.689841           0.000000                      0.0
3          2.382089           0.000000                      0.0
4          2.650065           2.721811                      0.0

   forecast_meter_rent_12m  forecast_price_energy_off_peak  \
0                 0.444045                        0.047073
1                 1.237292                        0.059075
2                 1.599009                        0.066622
3                 1.318689                        0.059448
4                 2.122969                        0.048014

   forecast_price_energy_peak  ...  months_modif_prod  months_renewal  \
0                    0.040659  ...                  2               6
1                    0.000000  ...                 76               4
2                    0.036589  ...                 68               8
3                    0.000000  ...                 69               9
4                    0.041399  ...                 71               9

   channel_sales_MISSING  channel_sales_ewpakwlliwisiwduibdlfmalxowmwpci  \
0                      0                                               0
1                      1                                               0
2                      0                                               0
3                      0                                               0
4                      1                                               0

   channel_sales_foosdfpfkusacimwkcsosbicdxkicaua  \
0                                               1
1                                               0
2                                               1
3                                               0
4                                               0

   channel_sales_lmkebamcaaclubfxadlmueccxoimlema  \
0                                               0
1                                               0
2                                               0
3                                               1
4                                               0

   channel_sales_usilxuppasemubllopkaafesmlibmsdf  \
0                                               0
1                                               0
2                                               0
3                                               0
4                                               0

   origin_up_kamkkxfxxuwbdslkwifmmcsiusiuosws  \
0                                           0
1                                           1
```

```
2                                                   1
3                                                   1
4                                                   1

   origin_up_ldkssxwpmemidmecebumciepifcamkci  \
0                                                   0
1                                                   0
2                                                   0
3                                                   0
4                                                   0

   origin_up_lxidpiddsbxsbosboudacockeimpuepw
0                                                   1
1                                                   0
2                                                   0
3                                                   0
4                                                   0

[5 rows x 65 columns]
```

Correlations

Creating new featuers requires trial and error and requires iteration. To determine if we are on the right track, we can check the correlation between our new features with our target variable of interest. We also want to check if features are highly correlated with each other (and leave only one in the model), since predictive models work best when the features are uncorrelated.

```
correlation = df.drop(columns=['id']).corr()

# Plot correlation
plt.figure(figsize=(45, 45))
sns.heatmap(
    correlation,
    xticklabels=correlation.columns.values,
    yticklabels=correlation.columns.values,
    annot=True,
    annot_kws={'size': 12}
)
# Axis ticks size
plt.xticks(fontsize=15)
plt.yticks(fontsize=15)
plt.show()
```



We will remove two variables that exhibit a high correlation with other independent features.

```
df = df.drop(columns=['num_years_antig', 'forecast_cons_year'])
df.head()
```

	id	cons_12m	cons_gas_12m	cons_last_month
0	24011ae4ebbe3035111d65fa7c15bc57	0.000000	4.739944	0.000000
1	d29c2c54acc38ff3c0614d0a653813dd	3.668479	0.000000	0.000000
2	764c75f661154dac3a6c254cd082ea7d	2.736397	0.000000	0.000000
3	bba03439a292a1e166f80264c16191cb	3.200029	0.000000	0.000000
4	149d57cf92fc41cf94415803a877cb4b	3.646011	0.000000	2.721811

```
   forecast_cons_12m  forecast_discount_energy  forecast_meter_rent_12m  \
0           0.000000                       0.0                 0.444045
1           2.280920                       0.0                 1.237292
2           1.689841                       0.0                 1.599009
3           2.382089                       0.0                 1.318689
4           2.650065                       0.0                 2.122969

   forecast_price_energy_off_peak  forecast_price_energy_peak  \
0                        0.047073                    0.040659
1                        0.059075                    0.000000
2                        0.066622                    0.036589
3                        0.059448                    0.000000
4                        0.048014                    0.041399

   forecast_price_pow_off_peak  ...  months_modif_prod  months_renewal  \
0                     1.619163  ...                  2               6
1                     1.656207  ...                 76               4
2                     1.656207  ...                 68               8
3                     1.656207  ...                 69               9
4                     1.619163  ...                 71               9

   channel_sales_MISSING  channel_sales_ewpakwlliwisiwduibdlfmalxowmwpci  \
0                      0                                              0
1                      1                                              0
2                      0                                              0
3                      0                                              0
4                      1                                              0

   channel_sales_foosdfpfkusacimwkcsosbicdxkicaua  \
0                                               1
1                                               0
2                                               1
3                                               0
4                                               0

   channel_sales_lmkebamcaaclubfxadlmueccxoimlema  \
0                                               0
1                                               0
2                                               0
3                                               1
4                                               0

   channel_sales_usilxuppasemubllopkaafesmlibmsdf  \
0                                               0
1                                               0
2                                               0
3                                               0
4                                               0
```

```
   origin_up_kamkkxfxxuwbdslkwifmmcsiusiuosws  \
0                                            0
1                                            1
2                                            1
3                                            1
4                                            1

   origin_up_ldkssxwpmemidmecebumciepifcamkci  \
0                                            0
1                                            0
2                                            0
3                                            0
4                                            0

   origin_up_lxidpiddsbxsbosboudacockeimpuepw
0                                            1
1                                            0
2                                            0
3                                            0
4                                            0

[5 rows x 63 columns]
```

Feature Engineering and Modelling

---

1. Import packages
2. Load data
3. Modelling

---

1. Import packages

```
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

import pandas as pd
import numpy as np
import seaborn as sns
from datetime import datetime
import matplotlib.pyplot as plt

# Shows plots in jupyter notebook
%matplotlib inline

# Set plot style
sns.set(color_codes=True)
```

---

2. Load data

```
df = pd.read_csv('./data_for_predictions.csv')
df.drop(columns=["Unnamed: 0"], inplace=True)
df.head()
```

```
                                 id  cons_12m  cons_gas_12m  cons_last_month
\
0  24011ae4ebbe3035111d65fa7c15bc57  0.000000      4.739944         0.000000
1  d29c2c54acc38ff3c0614d0a653813dd  3.668479      0.000000         0.000000
2  764c75f661154dac3a6c254cd082ea7d  2.736397      0.000000         0.000000
3  bba03439a292a1e166f80264c16191cb  3.200029      0.000000         0.000000
4  149d57cf92fc41cf94415803a877cb4b  3.646011      0.000000         2.721811

   forecast_cons_12m  forecast_discount_energy  forecast_meter_rent_12m  \
0           0.000000                       0.0                 0.444045
1           2.280920                       0.0                 1.237292
2           1.689841                       0.0                 1.599009
3           2.382089                       0.0                 1.318689
4           2.650065                       0.0                 2.122969

   forecast_price_energy_off_peak  forecast_price_energy_peak  \
0                        0.114481                    0.098142
```

```
1                       0.145711                    0.000000
2                       0.165794                    0.087899
3                       0.146694                    0.000000
4                       0.116900                    0.100015

   forecast_price_pow_off_peak  ...  months_modif_prod  months_renewal  \
0                    40.606701  ...                  2               6
1                    44.311378  ...                 76               4
2                    44.311378  ...                 68               8
3                    44.311378  ...                 69               9
4                    40.606701  ...                 71               9

   channel_MISSING  channel_ewpakwlliwisiwduibdlfmalxowmwpci  \
0                0                                         0
1                1                                         0
2                0                                         0
3                0                                         0
4                1                                         0

   channel_foosdfpfkusacimwkcsosbicdxkicaua  \
0                                         1
1                                         0
2                                         1
3                                         0
4                                         0

   channel_lmkebamcaaclubfxadlmueccxoimlema  \
0                                         0
1                                         0
2                                         0
3                                         1
4                                         0

   channel_usilxuppasemubllopkaafesmlibmsdf  \
0                                         0
1                                         0
2                                         0
3                                         0
4                                         0

   origin_up_kamkkxfxxuwbdslkwifmmcsiusiuosws  \
0                                           0
1                                           1
2                                           1
3                                           1
4                                           1

   origin_up_ldkssxwpmemidmecebumciepifcamkci  \
0                                           0
```

```
1                                                   0
2                                                   0
3                                                   0
4                                                   0

   origin_up_lxidpiddsbxsbosboudacockeimpuepw
0                                                   1
1                                                   0
2                                                   0
3                                                   0
4                                                   0

[5 rows x 63 columns]
```

---

3. Modelling

We now have a dataset containing features that we have engineered and we are ready to start training a predictive model. Remember, we only need to focus on training a `Random Forest` classifier.

```
from sklearn import metrics
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
```

Data sampling

The first thing we want to do is split our dataset into training and test samples. The reason why we do this, is so that we can simulate a real life situation by generating predictions for our test sample, without showing the predictive model these data points. This gives us the ability to see how well our model is able to generalise to new data, which is critical.

A typical % to dedicate to testing is between 20-30, for this example we will use a 75-25% split between train and test respectively.

```
# Make a copy of our data
train_df = df.copy()

# Separate target variable from independent variables
y = df['churn']
X = df.drop(columns=['id', 'churn'])
print(X.shape)
print(y.shape)
```

```
(14606, 61)
(14606,)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, ran
dom_state=42)
print(X_train.shape)
```

```
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

```
(10954, 61)
(10954,)
(3652, 61)
(3652,)
```

Model training

Once again, we are using a `Random Forest` classifier in this example. A Random Forest sits within the category of `ensemble` algorithms because internally the `Forest` refers to a collection of `Decision Trees` which are tree-based learning algorithms. As the data scientist, you can control how large the forest is (that is, how many decision trees you want to include).

The reason why an `ensemble` algorithm is powerful is because of the laws of averaging, weak learners and the central limit theorem. If we take a single decision tree and give it a sample of data and some parameters, it will learn patterns from the data. It may be overfit or it may be underfit, but that is now our only hope, that single algorithm.

With `ensemble` methods, instead of banking on 1 single trained model, we can train 1000's of decision trees, all using different splits of the data and learning different patterns. It would be like asking 1000 people to all learn how to code. You would end up with 1000 people with different answers, methods and styles! The weak learner notion applies here too, it has been found that if you train your learners not to overfit, but to learn weak patterns within the data and you have a lot of these weak learners, together they come together to form a highly predictive pool of knowledge! This is a real life application of many brains are better than 1.

Now instead of relying on 1 single decision tree for prediction, the random forest puts it to the overall views of the entire collection of decision trees. Some ensemble algorithms using a voting approach to decide which prediction is best, others using averaging.

As we increase the number of learners, the idea is that the random forest's performance should converge to its best possible solution.

Some additional advantages of the random forest classifier include:

- The random forest uses a rule-based approach instead of a distance calculation and so features do not need to be scaled
- It is able to handle non-linear parameters better than linear based models

On the flip side, some disadvantages of the random forest classifier include:

- The computational power needed to train a random forest on a large dataset is high, since we need to build a whole ensemble of estimators.
- Training time can be longer due to the increased complexity and size of thee ensemble

```
model = RandomForestClassifier(
    n_estimators=1000
)
model.fit(X_train, y_train)
```

```
RandomForestClassifier(n_estimators=1000)
```

The `scikit-learn` documentation: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html, has a lot of information about the algorithm and the parameters that you can use when training a model.

For this example, I am using `n_estimators` = 1000. This means that my random forest will consist of 1000 decision trees. There are many more parameters that you can fine-tune within the random forest and finding the optimal combinations of parameters can be a manual task of exploration, trial and error, which will not be covered during this notebook.

Evaluation

Now let's evaluate how well this trained model is able to predict the values of the test dataset.

We are going to use 3 metrics to evaluate performance:

- Accuracy = the ratio of correctly predicted observations to the total observations
- Precision = the ability of the classifier to not label a negative sample as positive
- Recall = the ability of the classifier to find all the positive samples

The reason why we are using these three metrics is because a simple accuracy is not always a good measure to use. To give an example, let's say you're predicting heart failures with patients in a hospital and there were 100 patients out of 1000 that did have a heart failure.

If you predicted 80 out of 100 (80%) of the patients that did have a heart failure correctly, you might think that you've done well! However, this also means that you predicted 20 wrong and what may the implications of predicting these remaining 20 patients wrong? Maybe they miss out on getting vital treatment to save their lives.

As well as this, what about the impact of predicting negative cases as positive (people not having heart failure being predicted that they did), maybe a high number of false positives means that resources get used up on thee wrong people and a lot of time is wasted when they could have been helping the real heart failure sufferers.

This is just an example, but it illustrates why other performance metrics are necessary such `Precision` and `Recall`, which are good measures to use in a classification scenario.

```
predictions = model.predict(X_test)
tn, fp, fn, tp = metrics.confusion_matrix(y_test, predictions).ravel()
```

```
y_test.value_counts()
```

```
0    3286
1     366
Name: churn, dtype: int64
```

```
print(f"True positives: {tp}")
print(f"False positives: {fp}")
print(f"True negatives: {tn}")
print(f"False negatives: {fn}\n")

print(f"Accuracy: {metrics.accuracy_score(y_test, predictions)}")
print(f"Precision: {metrics.precision_score(y_test, predictions)}")
print(f"Recall: {metrics.recall_score(y_test, predictions)}")
```

```
True positives: 18
False positives: 4
True negatives: 3282
False negatives: 348

Accuracy: 0.9036144578313253
Precision: 0.8181818181818182
Recall: 0.04918032786885246
```

Looking at these results there are a few things to point out:

Note: If you are running this notebook yourself, you may get slightly different answers!

- Within the test set about 10% of the rows are churners (churn = 1).
- Looking at the true negatives, we have 3282 out of 3286. This means that out of all the negative cases (churn = 0), we predicted 3282 as negative (hence the name True negative). This is great!
- Looking at the false negatives, this is where we have predicted a client to not churn (churn = 0) when in fact they did churn (churn = 1). This number is quite high at 348, we want to get the false negatives to as close to 0 as we can, so this would need to be addressed when improving the model.
- Looking at false positives, this is where we have predicted a client to churn when they actually didnt churn. For this value we can see there are 4 cases, which is great!
- With the true positives, we can see that in total we have 366 clients that churned in the test dataset. However, we are only able to correctly identify 18 of those 366, which is very poor.
- Looking at the accuracy score, this is very misleading! Hence the use of precision and recall is important. The accuracy score is high, but it does not tell us the whole story.
- Looking at the precision score, this shows us a score of 0.82 which is not bad, but could be improved.
- However, the recall shows us that the classifier has a very poor ability to identify positive samples. This would be the main concern for improving this model!

So overall, we're able to very accurately identify clients that do not churn, but we are not able to predict cases where clients do churn! What we are seeing is that a high % of clients

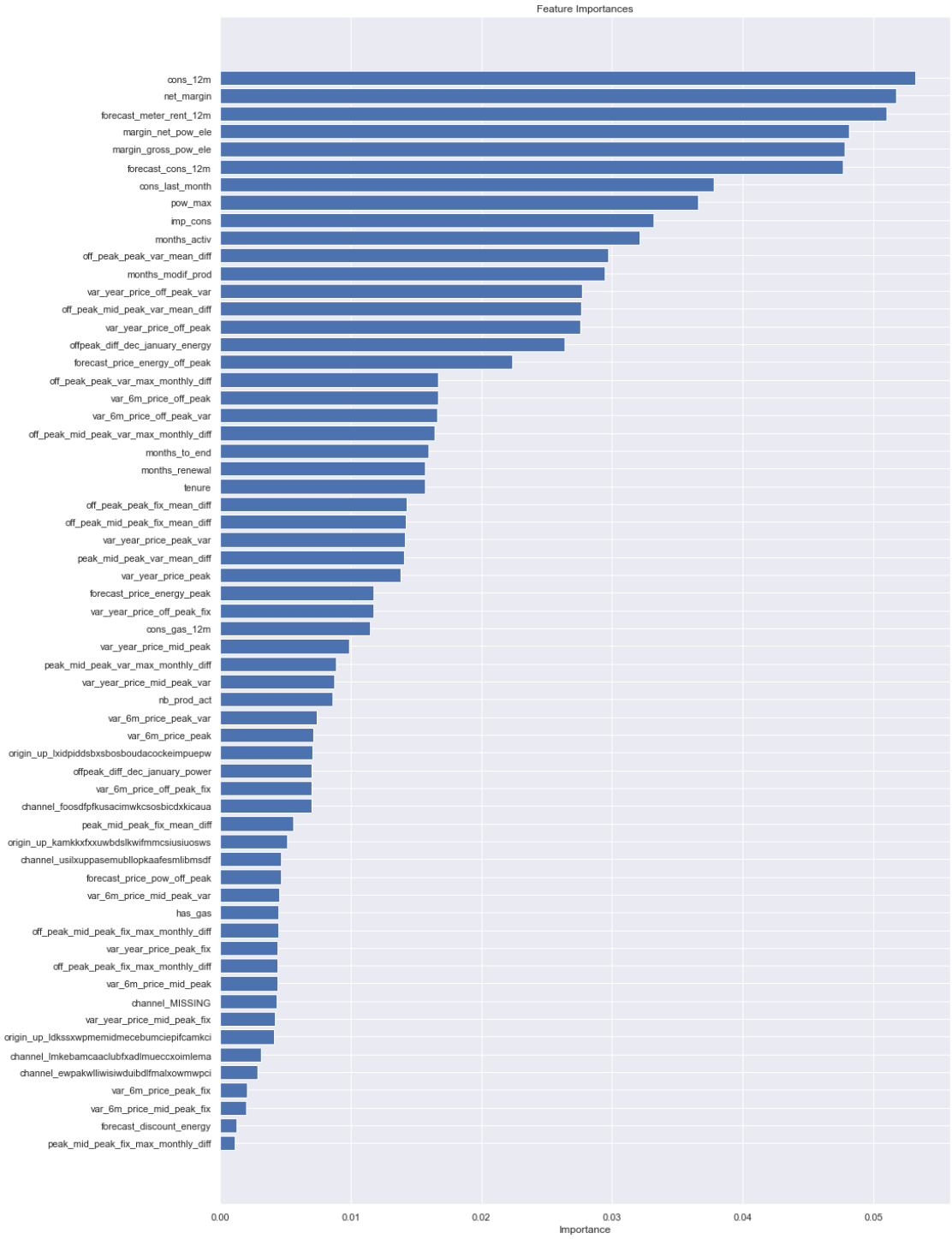
are being identified as not churning when they should be identified as churning. This in turn tells me that the current set of features are not discriminative enough to clearly distinguish between churners and non-churners.

A data scientist at this point would go back to feature engineering to try and create more predictive features. They may also experiment with optimising the parameters within the model to improve performance. For now, lets dive into understanding the model a little more.

Model understanding

A simple way of understanding the results of a model is to look at feature importances. Feature importances indicate the importance of a feature within the predictive model, there are several ways to calculate feature importance, but with the Random Forest classifier, we're able to extract feature importances using the built-in method on the trained model. In the Random Forest case, the feature importance represents the number of times each feature is used for splitting across all trees.

```
feature_importances = pd.DataFrame({
    'features': X_train.columns,
    'importance': model.feature_importances_
}).sort_values(by='importance', ascending=True).reset_index()

plt.figure(figsize=(15, 25))
plt.title('Feature Importances')
plt.barh(range(len(feature_importances)), feature_importances['importance'],
color='b', align='center')
plt.yticks(range(len(feature_importances)), feature_importances['features'])
plt.xlabel('Importance')
plt.show()
```



From this chart, we can observe the following points:

- Net margin and consumption over 12 months is a top driver for churn in this model
- Margin on power subscription also is an influential driver
- Time seems to be an influential factor, especially the number of months they have been active, their tenure and the number of months since they updated their contract

- The feature that our colleague recommended is in the top half in terms of how influential it is and some of the features built off the back of this actually outperform it
- Our price sensitivity features are scattered around but are not the main driver for a customer churning

The last observation is important because this relates back to our original hypothesis:

```
> Is churn driven by the customers' price sensitivity?
```

Based on the output of the feature importances, it is not a main driver but it is a weak contributor. However, to arrive at a conclusive result, more experimentation is needed.

```
proba_predictions = model.predict_proba(X_test)
probabilities = proba_predictions[:, 1]

X_test = X_test.reset_index()
X_test.drop(columns='index', inplace=True)

X_test['churn'] = predictions.tolist()
X_test['churn_probability'] = probabilities.tolist()
X_test.to_csv('out_of_sample_data_with_predictions.csv')
```